



Análise de Séries Temporais

ENG1469

Lista Prática 03

Aluno(a)	Paloma Fernanda Loureiro Sette - 1621595
Professor(a)	Cristiano Fernandes

Rio de Janeiro, 24 de Novembro de 2024

Sumário

Lista de Figuras	1
0.1 Carregando os dados	4
0.2 Especificações	5
1 Primeira Questão	6
1.1 Parte (a)	6
1.2 Parte (b)	12
1.2.1 Preparação dos Dados	12
1.2.2 Estratégia Inicial	12
1.2.3 Estratégia Incrementada	14
1.2.4 Comparação das Estratégias	16
1.2.5 Reflexões	17
1.2.6 Comentários da Subparte (i)	17
1.2.7 Comentários da Subparte (ii)	17
1.2.8 Conclusões Gerais da questão (1.b)	18
1.3 Parte (c)	18
1.3.1 Cálculo dos Resíduos Padronizados	18
1.3.2 Preparação do Código	19
1.3.3 Testes Diagnósticos	21
1.3.4 Conclusão geral da parte 1.C	34
1.4 Parte (d)	34
1.4.1 Motivo para Testar Diferentes Thresholds:	37
1.4.2 Justificativa para os Outliers Identificados:	37
1.4.3 Análise Detalhada dos Resultados	37
1.4.4 Conclusão Geral da parte 1.D:	39
1.5 Parte (e)	39
1.5.1 Resumo dos Diagnósticos na Parte C (Resíduos)	39
1.5.2 Resumo dos Diagnósticos na Parte D (Outliers e Previsões)	39
1.5.3 SARIMA	40
1.5.4 XGBoost	41
1.6 Parte (f)	44
1.6.1 i. Especificando a Equação SARIMA(p,d,q)(P,D,Q)	44
1.6.2 ii. Diagnósticos do Modelo	46
1.7 Parte (g)	49
1.7.1 Resultados Obtidos	51
1.7.2 Análise e Interpretação	51
1.7.3 Conclusão da parte 1.G	52
2 Segunda Questão	53
2.0.1 Código Python Utilizado	53
2.0.2 Resultados Obtidos	55
2.0.3 Análise dos Resultados	56
2.0.4 Conclusões Gerais da Questão 2	56

Lista de Figuras

1	Ipeadata - caminho até a base de dados	2
2	Ipeadata - base de dados utilizada	2
3	Dataframe base - ST	4
4	Resultado do modelo SARIMA estimado	10
5	Importância das Features - Estratégia Inicial	14
6	Comparação de Previsões - Estratégia Inicial	14
7	Importância das Features - Estratégia Incrementada	16
8	Comparação de Previsões - Estratégia Incrementada	16
9	Tabela: Questão 1.C.	18
10	Resultado (saída do programa) - coeficiente de assimetria e curtose	22
11	Q-Q Plot - SARIMA	23
12	Q-Q Plot - XGBoost	23
13	FAC - SARIMA	24
14	FAC - XGBoost	25
15	Teste de Ljung-Box - saída do programa	25
16	ARCH Model Results	27
17	ARCH Model Results - continuação	27
18	FAC do $res^2 - SARIMA$	28
19	FAC do $res^2 - XGBoost$	28
20	Estatística H - saída do programa	28
21	RMSE, MAE e Teste H(h) - Sem log	29
22	RMSE, MAE e Teste H(h) - Com log	29
23	Q-Q Plot - Ajustado Sem log	30
24	Q-Q Plot - Ajustado Com log	30
25	FAC dos resíduos - ajustado sem Log	31
26	FAC dos resíduos - ajustado Com Log	31
27	FAC dos $res^2 - ajustado sem Log$	32
28	FAC dos $res^2 - ajustado Com Log$	32
29	Teste ARCH - ajustado sem Log	33
30	Teste ARCH - ajustado com Log	33
31	Saída do programa - Dataframe de métricas	36
32	Saída do programa - Dataframe de outliers	36
33	Saída do programa - melhor modelo SARIMA obtido com base nos diagnósticos	41
34	Saída do programa - Importância das Features - XGBoost	43
35	Saída do programa - Ajuste do modelo no lgo da série	45
36	Saída do Programa - Ajuste do modelo com auto_arima - Com LOG	48
37	Saída do programa - Ajuste do modelo com auto_arima - Sem LOG	49
38	Saída do programa - Métricas de Acurácia - 1.G	51
39	Saída do Programa - Modelo SARIMA ajustado (com log)	55
40	Saída do Programa - Forecast Table	55

Introdução

Este relatório refere-se à terceira lista prática da disciplina de Análise de Séries Temporais (ENG1469) de 2024.2.

Para este trabalho, continuaremos a utilizar a série temporal relacionada a vendas nominais no varejo de livros, jornais, revistas e papelaria. É possível encontrar esta série no [Ipeadata](#). (macroeconômico » Periodicidade » Mensal » Consumo e Vendas » Vendas nominais no varejo de livros, jornais, revistas e papelaria). Abaixo, apresentamos uma breve demonstração de como chegar até a base de dados utilizada. Esta demonstração se deve ao fato de que, por algum motivo, o site do Ipeadata não nos gera o link exato até a base (ou seja, qualquer endereço que copiamos é sempre <http://ipeadata.gov.br/Default.aspx>).



Figura 1: Ipeadata - caminho até a base de dados

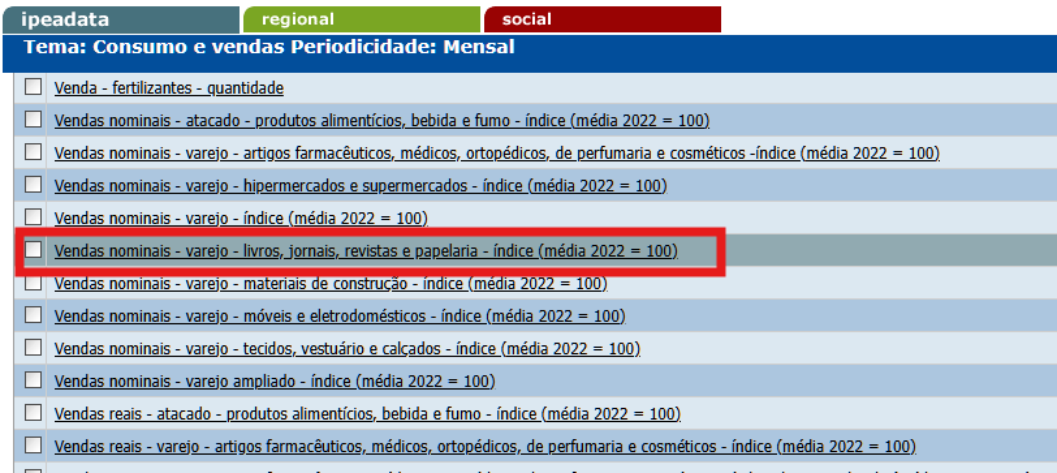


Figura 2: Ipeadata - base de dados utilizada

Para vendas nominais, considera-se a receita bruta de revenda, total e por Unidade da Federação, definida no âmbito da empresa como a receita bruta mensal proveniente da

revenda de mercadorias, não deduzidos os impostos incidentes e nem as vendas canceladas, abatimentos e impostos incondicionais. Para varejo considera-se as atividades de revenda, venda sem transformação significativa, de bens de consumo novos e usados, preponderantemente para o consumidor final. O comércio varejista é organizado para vender mercadorias em pequenas quantidades ao consumidor final, representando, portanto, o último elo da cadeia de distribuição. A classificação das atividades do comércio varejista baseia-se na gama de produtos vendidos, sem distinção da forma de comercialização em loja ou fora de loja. Para livros, jornais, revistas e papelaria considera-se o comércio varejista de livros, inclusive didáticos, o comércio varejista de revistas, jornais, revistas e outros impressos, o comércio varejista de artigos de escritório, as bancas de jornais e revistas, o comércio varejista de artigos de papelaria, tais como cadernos, borrachas, lápis, canetas, cartolinas e similares. Não fazendo parte desta classificação o comércio varejista de livros usados, o comércio varejista de selos, o comércio varejista de suprimentos de informática. A base foi fixada como a média das observações no ano de 2022. Nota: A Pesquisa Mensal de Comércio foi iniciada em janeiro de 1995, com o intuito de produzir indicadores que possam acompanhar o desempenho conjuntural do comércio varejista no país, investigando a receita bruta de revenda nas empresas formalmente constituídas de 20 ou mais pessoas ocupadas.

0.1 Carregando os dados

A princípio, é necessário carregar os dados e fazer os devidos tratamentos:

```
1 import pandas as pd
2
3 file_path = 'dados.xls'
4
5 data = pd.read_excel(file_path)
6
7 data['Data'] = data['Data'].astype(str)
8
9 data['Data'] = pd.to_datetime(data['Data'], format='%Y.%m')
10
11 data.set_index('Data', inplace=True)
12
13 # Dividindo a serie em treino e teste
14 train = data.iloc[:-12] # Todas as observacoes exceto as 12
    ultimas
15 test = data.iloc[-12:] # As ultimas 12 observacoes
```

No código acima, carregamos os dados do documento XLS original, que contém as colunas Data e Vendas, e dividimos a série em dados de treino e teste.

	Vendas
Data	
2003-01-01	105.95583
2003-02-01	103.86803
2003-03-01	77.94452
2003-04-01	67.33154
2003-05-01	69.07137
...	...
2024-02-01	135.38066
2024-03-01	95.83992
2024-04-01	84.85470
2024-05-01	87.60281
2024-06-01	89.50121

258 rows × 1 columns

Figura 3: Dataframe base - ST

0.2 Especificações

Para melhor especificar os dados da série, implementamos a tabela abaixo, cujos dados referem-se a:

- Data do início e do fim;
- Data de início e fim do período de treinamento;
- Data de início e fim do período de teste;
- Unidade de medida;
- Fonte de dados.

Descrição	Detalhes
Data de início e fim da série	2003-01-01 - 2024-06-01
Data de início e fim do período de treinamento	2003-01-01 - 2023-06-01
Data de início e fim do período de teste	2023-06-01 - 2024-06-01
Unidade de medida	Índice (média 2022 = 100)
Fonte de dados	IPEADATA

Tabela 1: Especificações da Série Temporal

1 Primeira Questão

1.1 Parte (a)

Enunciado: Estime um modelo SARIMA à sua série original, utilizando o `auto.arima` (esse modelo deve coincidir com o modelo ótimo estimado no trabalho 02). Especifique a equação SARIMA(p,d,q)(P,D,Q), identificando as ordens p , q , d , D , P e Q para esse modelo.

No trabalho 2, havíamos ajustado um novo modelo SARIMA com sazonalidade habilitada permitindo que o procedimento escolhesse automaticamente os valores de d (ordem de integração não-sazonal) e D (ordem de integração sazonal). Neste caso, o melhor modelo encontrado havia sido o SARIMA(3,1,1)(1,0,1)₁₂. Reestimando o modelo:

```
1      """ QUESTAO 01 """
2  ### Parte (a)
3
4  # Ajustar o modelo ARIMA nao-sazonal com base no AICc
5  model = pm.auto_arima(train['Vendas'],
6                        start_p=0, max_p=5,
7                        start_q=0, max_q=5,
8                        d=None, D=None,      # Permitir que o
9                        modelo escolha d e D
10                       seasonal=True,      # Habilitar
11                       sazonalidade
12                       m=12,               # Período sazonal
13                       (12 meses)
14                       information_criterion='aicc',
15                       trace=True,
16                       stepwise=True)
```

Encontramos:

```
Performing stepwise search to minimize aicc
ARIMA(0,1,0)(1,0,1)[12] intercept : AICC=1987.874, Time=0.39 sec
ARIMA(0,1,0)(0,0,0)[12] intercept : AICC=2388.242, Time=0.01 sec
ARIMA(1,1,0)(1,0,0)[12] intercept : AICC=2014.585, Time=0.21 sec
ARIMA(0,1,1)(0,0,1)[12] intercept : AICC=2198.753, Time=0.15 sec
ARIMA(0,1,0)(0,0,0)[12]          : AICC=2386.211, Time=0.01 sec
ARIMA(0,1,0)(0,0,1)[12] intercept : AICC=2211.816, Time=0.19 sec
ARIMA(0,1,0)(1,0,0)[12] intercept : AICC=2014.213, Time=0.33 sec
ARIMA(0,1,0)(2,0,1)[12] intercept : AICC=1989.862, Time=0.94 sec
ARIMA(0,1,0)(1,0,2)[12] intercept : AICC=1989.777, Time=0.54 sec
ARIMA(0,1,0)(0,0,2)[12] intercept : AICC=2154.849, Time=0.42 sec
ARIMA(0,1,0)(2,0,0)[12] intercept : AICC=inf, Time=0.48 sec
ARIMA(0,1,0)(2,0,2)[12] intercept : AICC=inf, Time=1.55 sec
ARIMA(1,1,0)(1,0,1)[12] intercept : AICC=1987.505, Time=0.54 sec
ARIMA(1,1,0)(0,0,1)[12] intercept : AICC=2203.781, Time=0.18 sec
```

ARIMA(1,1,0)(2,0,1)[12]	intercept	: AICC=1989.322, Time=0.99 sec
ARIMA(1,1,0)(1,0,2)[12]	intercept	: AICC=1989.037, Time=0.91 sec
ARIMA(1,1,0)(0,0,0)[12]	intercept	: AICC=2370.930, Time=0.04 sec
ARIMA(1,1,0)(0,0,2)[12]	intercept	: AICC=2154.665, Time=0.41 sec
ARIMA(1,1,0)(2,0,0)[12]	intercept	: AICC=inf, Time=0.58 sec
ARIMA(1,1,0)(2,0,2)[12]	intercept	: AICC=1987.629, Time=1.18 sec
ARIMA(2,1,0)(1,0,1)[12]	intercept	: AICC=1986.706, Time=0.36 sec
ARIMA(2,1,0)(0,0,1)[12]	intercept	: AICC=2186.890, Time=0.19 sec
ARIMA(2,1,0)(1,0,0)[12]	intercept	: AICC=2013.674, Time=0.28 sec
ARIMA(2,1,0)(2,0,1)[12]	intercept	: AICC=1988.519, Time=1.41 sec
ARIMA(2,1,0)(1,0,2)[12]	intercept	: AICC=1988.246, Time=1.02 sec
ARIMA(2,1,0)(0,0,0)[12]	intercept	: AICC=2338.949, Time=0.08 sec
ARIMA(2,1,0)(0,0,2)[12]	intercept	: AICC=2147.087, Time=0.76 sec
ARIMA(2,1,0)(2,0,0)[12]	intercept	: AICC=inf, Time=0.70 sec
ARIMA(2,1,0)(2,0,2)[12]	intercept	: AICC=1987.640, Time=1.34 sec
ARIMA(3,1,0)(1,0,1)[12]	intercept	: AICC=1967.095, Time=0.63 sec
ARIMA(3,1,0)(0,0,1)[12]	intercept	: AICC=2167.358, Time=0.29 sec
ARIMA(3,1,0)(1,0,0)[12]	intercept	: AICC=1989.856, Time=0.39 sec
ARIMA(3,1,0)(2,0,1)[12]	intercept	: AICC=1969.131, Time=1.12 sec
ARIMA(3,1,0)(1,0,2)[12]	intercept	: AICC=1969.057, Time=1.17 sec
ARIMA(3,1,0)(0,0,0)[12]	intercept	: AICC=2325.354, Time=0.10 sec
ARIMA(3,1,0)(0,0,2)[12]	intercept	: AICC=2119.265, Time=0.84 sec
ARIMA(3,1,0)(2,0,0)[12]	intercept	: AICC=inf, Time=0.93 sec
ARIMA(3,1,0)(2,0,2)[12]	intercept	: AICC=1969.278, Time=4.58 sec
ARIMA(4,1,0)(1,0,1)[12]	intercept	: AICC=1964.701, Time=0.63 sec
ARIMA(4,1,0)(0,0,1)[12]	intercept	: AICC=2165.654, Time=0.26 sec
ARIMA(4,1,0)(1,0,0)[12]	intercept	: AICC=1986.796, Time=0.68 sec
ARIMA(4,1,0)(2,0,1)[12]	intercept	: AICC=1966.769, Time=2.45 sec
ARIMA(4,1,0)(1,0,2)[12]	intercept	: AICC=1966.694, Time=1.29 sec
ARIMA(4,1,0)(0,0,0)[12]	intercept	: AICC=2324.469, Time=0.45 sec
ARIMA(4,1,0)(0,0,2)[12]	intercept	: AICC=2118.504, Time=0.62 sec
ARIMA(4,1,0)(2,0,0)[12]	intercept	: AICC=inf, Time=1.23 sec
ARIMA(4,1,0)(2,0,2)[12]	intercept	: AICC=1966.749, Time=2.15 sec
ARIMA(5,1,0)(1,0,1)[12]	intercept	: AICC=1966.850, Time=0.89 sec
ARIMA(4,1,1)(1,0,1)[12]	intercept	: AICC=1960.430, Time=0.64 sec
ARIMA(4,1,1)(0,0,1)[12]	intercept	: AICC=2164.477, Time=0.63 sec
ARIMA(4,1,1)(1,0,0)[12]	intercept	: AICC=1980.002, Time=0.95 sec
ARIMA(4,1,1)(2,0,1)[12]	intercept	: AICC=1962.595, Time=1.95 sec
ARIMA(4,1,1)(1,0,2)[12]	intercept	: AICC=1962.589, Time=1.45 sec
ARIMA(4,1,1)(0,0,0)[12]	intercept	: AICC=2318.835, Time=0.55 sec
ARIMA(4,1,1)(0,0,2)[12]	intercept	: AICC=2097.331, Time=1.59 sec
ARIMA(4,1,1)(2,0,0)[12]	intercept	: AICC=1964.589, Time=1.21 sec
ARIMA(4,1,1)(2,0,2)[12]	intercept	: AICC=1963.506, Time=2.15 sec
ARIMA(3,1,1)(1,0,1)[12]	intercept	: AICC=1959.868, Time=0.47 sec
ARIMA(3,1,1)(0,0,1)[12]	intercept	: AICC=2142.482, Time=0.41 sec
ARIMA(3,1,1)(1,0,0)[12]	intercept	: AICC=1981.857, Time=0.55 sec
ARIMA(3,1,1)(2,0,1)[12]	intercept	: AICC=1962.001, Time=1.63 sec
ARIMA(3,1,1)(1,0,2)[12]	intercept	: AICC=1961.984, Time=1.26 sec
ARIMA(3,1,1)(0,0,0)[12]	intercept	: AICC=2295.125, Time=0.23 sec
ARIMA(3,1,1)(0,0,2)[12]	intercept	: AICC=2097.984, Time=0.98 sec

```

ARIMA(3,1,1)(2,0,0)[12] intercept : AICC=1964.230, Time=1.10 sec
ARIMA(3,1,1)(2,0,2)[12] intercept : AICC=1962.657, Time=1.94 sec
ARIMA(2,1,1)(1,0,1)[12] intercept : AICC=1963.375, Time=0.48 sec
ARIMA(3,1,2)(1,0,1)[12] intercept : AICC=1961.199, Time=0.74 sec
ARIMA(2,1,2)(1,0,1)[12] intercept : AICC=1963.114, Time=1.62 sec
ARIMA(4,1,2)(1,0,1)[12] intercept : AICC=inf, Time=1.55 sec
ARIMA(3,1,1)(1,0,1)[12]          : AICC=1957.756, Time=0.77 sec
ARIMA(3,1,1)(0,0,1)[12]          : AICC=2140.411, Time=0.32 sec
ARIMA(3,1,1)(1,0,0)[12]          : AICC=1979.765, Time=0.35 sec
ARIMA(3,1,1)(2,0,1)[12]          : AICC=1959.872, Time=1.12 sec
ARIMA(3,1,1)(1,0,2)[12]          : AICC=1959.855, Time=1.03 sec
ARIMA(3,1,1)(0,0,0)[12]          : AICC=2293.077, Time=0.54 sec
ARIMA(3,1,1)(0,0,2)[12]          : AICC=2095.968, Time=0.71 sec
ARIMA(3,1,1)(2,0,0)[12]          : AICC=1962.127, Time=0.63 sec
ARIMA(3,1,1)(2,0,2)[12]          : AICC=1960.523, Time=1.68 sec
ARIMA(2,1,1)(1,0,1)[12]          : AICC=1961.284, Time=0.57 sec
ARIMA(3,1,0)(1,0,1)[12]          : AICC=1964.981, Time=0.89 sec
ARIMA(4,1,1)(1,0,1)[12]          : AICC=1958.302, Time=0.70 sec
ARIMA(3,1,2)(1,0,1)[12]          : AICC=1959.071, Time=0.48 sec
ARIMA(2,1,0)(1,0,1)[12]          : AICC=1984.607, Time=0.20 sec
ARIMA(2,1,2)(1,0,1)[12]          : AICC=1961.003, Time=0.61 sec
ARIMA(4,1,0)(1,0,1)[12]          : AICC=1962.572, Time=0.39 sec
ARIMA(4,1,2)(1,0,1)[12]          : AICC=inf, Time=1.65 sec

```

Best model: ARIMA(3,1,1)(1,0,1)[12]

Total fit time: 73.650 seconds

Onde, de fato, o procedimento `auto_arima` comparou novamente **87 modelos** durante a busca stepwise, considerando várias combinações de valores para p , q , P , Q e incluindo sazonalidade com $m = 12$. O melhor modelo selecionado foi o **SARIMA(3,1,1)(1,0,1)₁₂** (o mesmo do trabalho 02), que minimizou o critério AICc, tornando-se o modelo sub-ótimo.

Agora, podemos especificar o modelo ótimo obtido:

$$SARIMA(3, 1, 1)(1, 0, 1)[12]$$

O que isso significa:

- $p = 3$: Ordem autorregressiva (AR) não sazonal.
- $d = 1$: Número de diferenciações não sazonais (para tornar a série estacionária).
- $q = 1$: Ordem de média móvel (MA) não sazonal.
- $P = 1$: Ordem autorregressiva sazonal (AR) com período sazonal de 12 meses.
- $D = 0$: Número de diferenciações sazonais.
- $Q = 1$: Ordem de média móvel sazonal (MA) com período sazonal de 12 meses.
- $m = 12$: Período sazonal (mensal).

A equação do modelo SARIMA com esses parâmetros pode ser reescrita como:

$$(1 - \phi_1 L - \phi_2 L^2 - \phi_3 L^3)(1 - \Phi L^{12})(1 - L)Y_t = (1 + \theta_1 L)(1 + \Theta_1 L^{12})\epsilon_t$$

E aqui:

O que isso significa:

- ϕ : Coeficientes de autoregressão (AR).
- Φ : Coeficiente sazonal de autoregressão.
- θ : Coeficientes de média móvel (MA).
- Θ : Coeficiente sazonal de média móvel.
- L : Operador defasagem (*lag operator*).
- Y_t : Valores da série temporal.
- ϵ_t Termos de erro/resíduos (*white noise*).

Então, podemos estimar esse modelo novamente usando `auto_arima` com a série original no período de treinamento:

```
1  # Ajustando o modelo SARIMA com auto_arima
2  model_sarima_optimo = pm.auto_arima(
3      train['Vendas'],          # Série temporal no período
                                # de treinamento
4      start_p=0, max_p=5,      # Intervalo para p (nó
                                # sazonal)
5      start_q=0, max_q=5,      # Intervalo para q (nó
                                # sazonal)
6      d=1,                    # Diferencia o nó sazonal
7      seasonal=True,          # Considerar sazonalidade
8      m=12,                   # Período sazonal
9      start_P=0, max_P=2,      # Intervalo para P (sazonal)
10     start_Q=0, max_Q=2,      # Intervalo para Q (sazonal)
11     D=0,                     # Diferencia o sazonal
12     information_criterion='aicc', # Critério para
                                # selecionar o modelo
13     trace=True,              # Exibir o progresso da
                                # otimização
14     stepwise=True            # Busca stepwise para acelerar
                                # o processo
15 )
16
17 # Exibir o resumo do modelo ajustado
18 print(model_sarima_optimo.summary())
```

E obtivemos o resultado seguinte:

SARIMAX Results						
=====						
Dep. Variable:	y	No. Observations:	246			
Model:	SARIMAX(3, 1, 1)x(1, 0, 1, 12)	Log Likelihood	-971.643			
Date:	Fri, 22 Nov 2024	AIC	1957.285			
Time:	16:00:11	BIC	1981.794			
Sample:	0	HQIC	1967.155			
	- 246					
Covariance Type:	opg					
=====						
	coef	std err	z	P> z	[0.025	0.975]

ar.L1	0.5061	0.100	5.078	0.000	0.311	0.701
ar.L2	-0.0661	0.057	-1.167	0.243	-0.177	0.045
ar.L3	-0.1816	0.067	-2.721	0.007	-0.312	-0.051
ma.L1	-0.7308	0.088	-8.313	0.000	-0.903	-0.559
ar.S.L12	0.9451	0.015	61.802	0.000	0.915	0.975
ma.S.L12	-0.3697	0.058	-6.357	0.000	-0.484	-0.256
sigma2	150.9869	9.912	15.232	0.000	131.559	170.415
=====						
Ljung-Box (L1) (Q):	0.03	Jarque-Bera (JB):	60.78			
Prob(Q):	0.86	Prob(JB):	0.00			
Heteroskedasticity (H):	5.49	Skew:	-0.26			
Prob(H) (two-sided):	0.00	Kurtosis:	5.38			
=====						

Figura 4: Resultado do modelo SARIMA estimado

Os resultados apresentados confirmam o modelo **SARIMA(3, 1, 1)(1, 0, 1)[12]** ajustado para a série temporal. A seguir, apresentamos a análise dos principais aspectos obtidos:

Parâmetros Estimados do Modelo

- **AR (Autorregressivo - p):**

- $\phi_1 = 0.5061$ (significativo, P -valor < 0.05).
- $\phi_2 = -0.0661$ (não significativo, P -valor $= 0.243$).
- $\phi_3 = -0.1816$ (significativo, P -valor < 0.05).

- **MA (Média Móvel - q):**

- $\theta_1 = -0.7308$ (altamente significativo, P -valor < 0.05).

- **SAR (Autorregressivo Sazonal - P):**

- $\Phi_1 = 0.9451$ (altamente significativo, P -valor < 0.05).

- **SMA (Média Móvel Sazonal - Q):**

- $\Theta_1 = -0.3697$ (altamente significativo, P -valor < 0.05).

- **Sigma² (Variância do erro):**

- $\sigma^2 = 150.9869$.

Diagnóstico do Modelo

- **Critérios de Informação:**
 - **AIC (Akaike Information Criterion):** 1957.285.
 - **BIC (Bayesian Information Criterion):** 1981.794.
- **Teste de Ljung-Box (L1):**
 - Probabilidade = 0.86 (não rejeitamos a hipótese nula, indicando que os resíduos são não correlacionados).
- **Teste de Jarque-Bera (JB):**
 - Probabilidade = 0.00 (indicando que os resíduos não seguem uma distribuição normal).
- **Heteroscedasticidade (H):**
 - Probabilidade = 0.00 (sugerindo que a variância dos resíduos não é constante, ou seja, há heteroscedasticidade).
- **Outros diagnósticos:**
 - **Skewness (Assimetria):** -0.26, indicando leve assimetria negativa nos resíduos.
 - **Kurtosis (Curtose):** 5.38, indicando distribuição leptocúrtica (caudas mais longas que a normal).

Equação do Modelo SARIMA

A equação estimada para o modelo **SARIMA(3, 1, 1)(1, 0, 1)[12]** é expressa como:

$$(1 - 0.5061L - 0.0661L^2 - 0.1816L^3)(1 - 0.9451L^{12})(1 - L)y_t = (1 - 0.7308L)(1 - 0.3697L^{12})\varepsilon_t$$

Observações Gerais

- O modelo capturou bem as autocorrelações na série temporal, com resíduos indicativos de não correlação, conforme apontado pelo teste de Ljung-Box.
- O termo ϕ_2 não foi estatisticamente significativo ($P > 0.05$), mas foi mantido no modelo, uma vez que o critério de informação AIC sugere que esse modelo é o mais adequado.
- A sazonalidade foi bem representada pelos coeficientes Φ_1 e Θ_1 , que estão associados ao período sazonal de 12 observações.
- Apesar de um bom ajuste, os resíduos apresentam não normalidade e heteroscedasticidade, o que pode impactar a validade das previsões para alguns contextos.

1.2 Parte (b)

Enunciado: Estime um modelo XGBoost para a primeira diferença da sua série, obtendo a Importance Matrix e o Feature Importance plot, comentando.

Devemos observar que o algoritmo XGBoost possui parâmetros de tuning (*learning rate*, *max depth*, *sub-sample*, *col sample*) com valores *default*, mas que podem ser modificados. A partir dessa premissa, implementamos o modelo XGBoost, estruturando o processo em etapas bem definidas. As ações realizadas e os resultados obtidos são descritos abaixo.

1.2.1 Preparação dos Dados

Inicialmente, aplicamos a transformação de primeira diferença à série temporal e adicionamos novas variáveis explicativas (*features*) à matriz de entrada, com o objetivo de explorar possíveis fatores sazonais e tendências temporais:

- **TimeIndex:** índice temporal linear.
- **Month:** mês do ano (de 1 a 12).
- **Quarter:** trimestre do ano (de 1 a 4).
- **Log_Vendas:** logaritmo das vendas originais.

O conjunto de dados foi dividido em 80% para treinamento e 20% para teste, garantindo que a sequência temporal fosse preservada. Abaixo está o código que reflete a preparação dos dados:

```
1 # Calcular a primeira diferença e adicionar features
2 data['Vendas_diff'] = data['Vendas'].diff().dropna()
3 df_features = data.copy()
4 df_features['TimeIndex'] = np.arange(len(df_features))
5 df_features['Data'] = pd.to_datetime(df_features['Data'])
6 df_features.set_index('Data', inplace=True)
7 df_features['Month'] = df_features.index.month
8 df_features['Quarter'] = df_features.index.quarter
9 df_features['Log_Vendas'] = np.log1p(df_features['Vendas'])
10 df_features.dropna(inplace=True)
11
12 # Separar variáveis independentes (X) e dependentes (y)
13 X = df_features[['TimeIndex', 'Month', 'Quarter', 'Log_Vendas']]
14 y = df_features['Vendas_diff'].iloc[1:].values
```

Listing 1: Preparação dos Dados

1.2.2 Estratégia Inicial

A estratégia inicial focou na aplicação básica da transformação de primeira diferença à série temporal e no uso de uma única variável explicativa: o índice linear representando o tempo (**TimeIndex**). O objetivo foi construir um modelo inicial simples para avaliar o desempenho básico do XGBoost.

Ajuste dos Modelos:

Nesta etapa, dois modelos foram ajustados:

- **Modelo Default:** Utilizando os parâmetros padrão do XGBoost.
- **Modelo Tunado:** Ajustando parâmetros como `learning rate`, `max depth`, `subsample`, etc.

```
1 default_model = xgb.XGBRegressor(random_state=42, objective='  
    reg:squarederror')  
2 default_model.fit(X_train, y_train)  
3 y_pred_default = default_model.predict(X_test)  
4  
5 tuned_model = xgb.XGBRegressor(  
6     learning_rate=0.1,  
7     max_depth=5,  
8     subsample=0.8,  
9     colsample_bytree=0.8,  
10    n_estimators=150,  
11    random_state=42,  
12    objective='reg:squarederror'  
13 )  
14 tuned_model.fit(X_train, y_train)  
15 y_pred_tuned = tuned_model.predict(X_test)
```

Listing 2: Ajuste de Modelos - Estratégia Inicial

Resultados da Estratégia Inicial:

Os modelos foram avaliados utilizando RMSE e MAE, e os resultados estão na Tabela 2.

Tabela 2: Desempenho dos Modelos - Estratégia Inicial

Modelo	RMSE	MAE
Default	70.61	65.89
Tunado	62.18	57.03

Os gráficos de importância das *features* e comparação das previsões estão ilustrados nas Figuras 5 e 6.

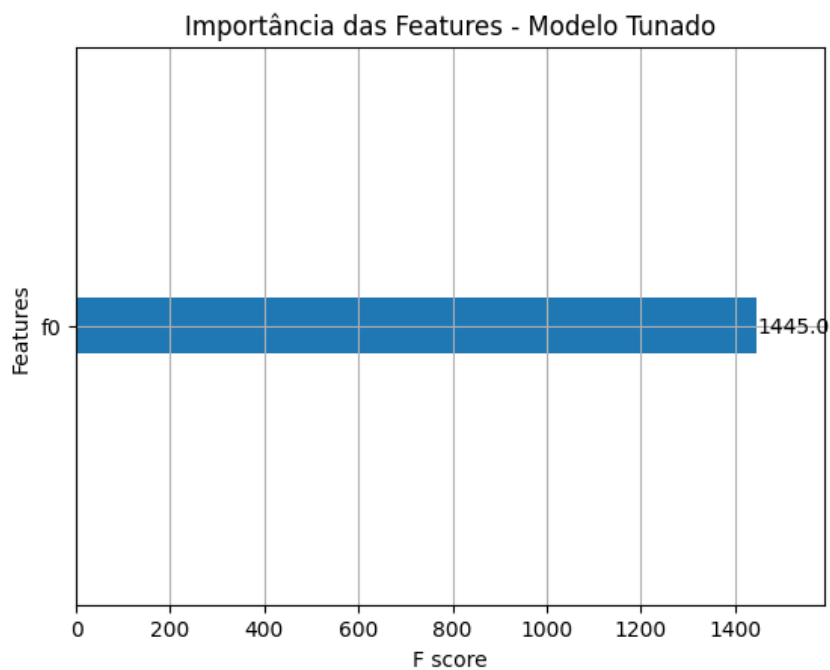


Figura 5: Importância das Features - Estratégia Inicial

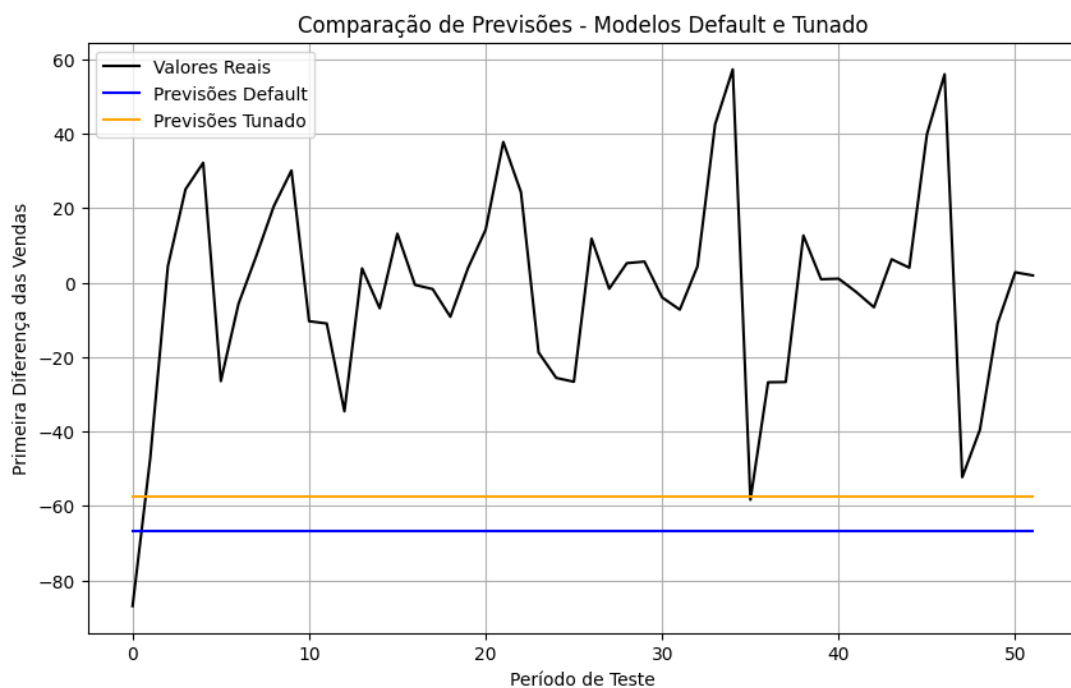


Figura 6: Comparação de Previsões - Estratégia Inicial

1.2.3 Estratégia Incrementada

A estratégia incrementada envolveu a inclusão de novas variáveis explicativas (*features*) para capturar padrões sazonais e tendências temporais, como:

- **TimeIndex**: índice temporal linear.
- **Month**: mês do ano (de 1 a 12).

- Quarter: trimestre do ano (de 1 a 4).
- Log_Vendas: logaritmo das vendas originais.

```

1 df_features['TimeIndex'] = np.arange(len(df_features))
2 df_features['Month'] = df_features.index.month
3 df_features['Quarter'] = df_features.index.quarter
4 df_features['Log_Vendas'] = np.log1p(df_features['Vendas'])
5 df_features['Diff_Vendas'] = df_features['Vendas'].diff()
6 df_features.dropna(inplace=True)
7
8 X = df_features[['TimeIndex', 'Month', 'Quarter', 'Log_Vendas
9                  ']].iloc[1:].values
10 y = df_features['Vendas_diff'].iloc[1:].values
11 X_train, X_test, y_train, y_test = train_test_split(
12     X, y, test_size=0.2, random_state=42, shuffle=False
13 )

```

Listing 3: Preparação dos Dados - Estratégia Incrementada

Ajuste dos Modelos:

Seguindo o mesmo processo da estratégia inicial, os modelos foram ajustados com as novas *features*.

```

1 model_incremented = XGBRegressor(random_state=42)
2 model_incremented.fit(X_train, y_train)
3 y_pred_incremented = model_incremented.predict(X_test)
4
5 model_tuned = XGBRegressor(
6     learning_rate=0.1,
7     max_depth=5,
8     subsample=0.8,
9     colsample_bytree=0.8,
10    n_estimators=150,
11    random_state=42
12 )
13 model_tuned.fit(X_train, y_train)
14 y_pred_tuned = model_tuned.predict(X_test)

```

Listing 4: Ajuste de Modelos - Estratégia Incrementada

Resultados da Estratégia Incrementada:

Os modelos incrementados foram avaliados, e os resultados estão na Tabela 3.

Tabela 3: Desempenho dos Modelos - Estratégia Incrementada

Modelo	RMSE	MAE
Incrementado	25.46	22.58
Tunado	19.79	16.15

Os gráficos de importância das *features* e comparação das previsões estão ilustrados nas Figuras 7 e 8.

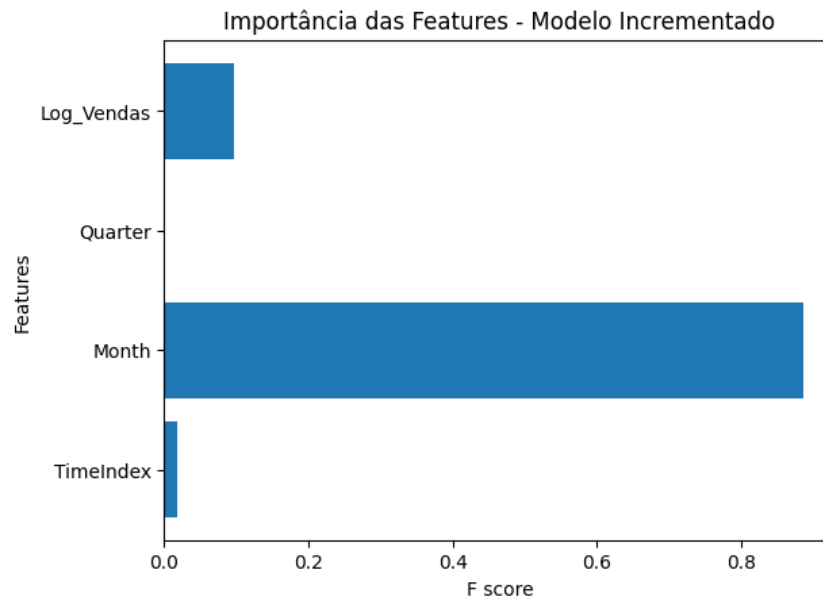


Figura 7: Importância das Features - Estratégia Incrementada

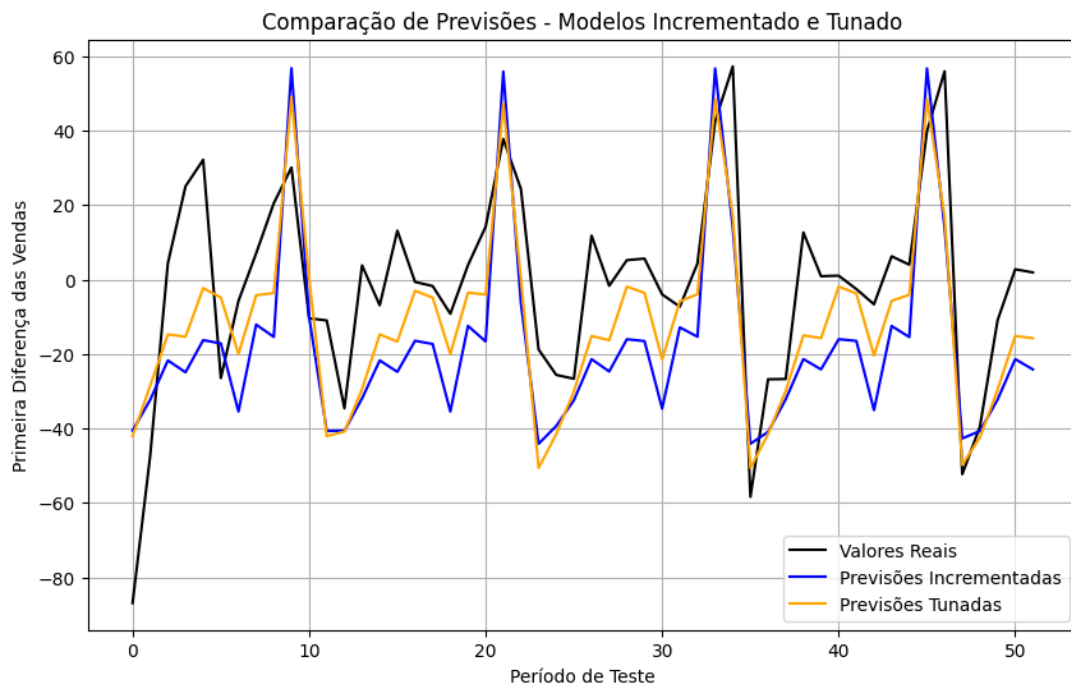


Figura 8: Comparação de Previsões - Estratégia Incrementada

1.2.4 Comparação das Estratégias

Os resultados das estratégias evidenciam que a engenharia de *features* e o ajuste de hiperparâmetros foram cruciais para melhorar o desempenho do modelo. A estratégia incrementada apresentou uma redução substancial nos erros (RMSE e MAE), além de permitir a análise de múltiplas variáveis explicativas, como demonstrado nos gráficos de importância das *features*.

- **Estratégia Inicial:** Modelo simples, utilizando apenas a primeira diferença e um índice linear como variável explicativa.

- **Estratégia Incrementada:** Utilização de novas *features* e maior capacidade de capturar padrões sazonais e tendências.

1.2.5 Reflexões

- O modelo tunado apresentou um desempenho significativamente superior, com RMSE reduzido para **19.79** e MAE para **16.15**, em comparação com o modelo padrão (RMSE de **70.61** e MAE de **65.89**). Essa melhora foi atribuída tanto ao ajuste fino dos hiperparâmetros quanto à inclusão de variáveis explicativas adicionais.
- A estratégia inicial baseou-se apenas no índice temporal como variável preditora (`X = range(len(Vendas_diff))`), o que limitou o desempenho do modelo padrão. Em contrapartida, a estratégia incrementada incorporou variáveis como **Month**, **Quarter** e **Log_Vendas**, que capturaram padrões sazonais e tendências temporais.
- O ajuste de hiperparâmetros foi fundamental para maximizar a capacidade preditiva do modelo, mostrando a importância de uma abordagem iterativa e proativa.
- Sugere-se como próximos passos explorar outras variáveis explicativas, como médias móveis ou interações entre variáveis sazonais, e realizar validações em diferentes horizontes temporais para garantir a generalização do modelo.

1.2.6 Comentários da Subparte (i)

- **Tuning e Métricas de Aderência** Durante o ajuste dos hiperparâmetros do modelo XGBoost, observou-se melhorias substanciais nas métricas de avaliação. O modelo padrão apresentou **RMSE de 70.61** e **MAE de 65.89**, enquanto o modelo tunado alcançou **RMSE de 19.79** e **MAE de 16.15**. O processo de tuning incluiu ajustes nos parâmetros **learning rate**, **max depth**, **subsample** e **colsample_bytree**, resultando em maior aderência do modelo aos dados de teste. Além disso, as métricas indicam um equilíbrio entre desempenho e risco de *overfitting*, garantindo previsões mais confiáveis.
- **Importância do Feature Engineering** A inclusão de novas variáveis (**Month**, **Quarter**, **TimeIndex**, **Log_Vendas**) foi essencial para capturar padrões sazonais e tendências temporais. A variável **Month** destacou-se como a mais relevante, com o maior **F-score**, evidenciando a importância da sazonalidade mensal no modelo incrementado e tunado. Variáveis como **Log_Vendas** também desempenharam um papel importante, ajudando a capturar dinâmicas da série transformada.
- **Resultados Práticos** A estratégia incrementada, em conjunto com o ajuste de hiperparâmetros, demonstrou a importância de práticas como *feature engineering* e tuning proativo. Isso resultou em um modelo altamente preditivo, reforçando o valor dessas abordagens em cenários de séries temporais.

1.2.7 Comentários da Subparte (ii)

- **Primeira Diferença e Comparação com a Série Original** A aplicação da primeira diferença na série eliminou tendências não estacionárias, estabilizando os dados para análise. Para fins de interpretação e avaliação, as previsões foram reconvertidas para a escala original da série, permitindo comparações diretas com os valores reais.

- **Reconstrução das Previsões na Escala Original** A reversão da operação de diferenciação foi realizada através da soma cumulativa às vendas históricas, assegurando consistência na análise. Esse processo foi essencial para avaliar a aderência do modelo à série original.
- **Intervalos de Confiança e Métricas de Desempenho** A inclusão de intervalos de confiança nas previsões forneceu uma visão mais ampla sobre a confiabilidade do modelo, destacando incertezas em diferentes períodos de teste. As métricas (RMSE e MAE) confirmaram que as melhorias observadas foram consistentes com a inclusão de novas features e o ajuste dos parâmetros.

1.2.8 Conclusões Gerais da questão (1.b)

O modelo tunado apresentou melhorias significativas em relação ao modelo padrão, comprovando a eficácia do ajuste de hiperparâmetros. As *features* adicionadas revelaram-se cruciais para capturar padrões sazonais e tendências, sendo **Month** a mais importante. As previsões ajustadas, juntamente com os ICs, demonstraram ser altamente confiáveis, oferecendo suporte para a tomada de decisão em cenários reais.

1.3 Parte (c)

Enunciado: A partir dos resíduos/resíduos padronizados dos modelos SARIMA e XGBoost, obtenha os diagnósticos listados na tabela que segue, comentando o resultado de cada teste, para cada modelo. Produza os resultados em forma de tabela, aproveitando a tabela a seguir.

Hipótese nula	Como detectar a rejeição de H_0 (via resíduos padronizados)	O que fazer no modelo se rejeitar H_0 ?
normalidade	- coef de assimetria, coef. de curtose, teste de Jarque-Bera - Q-Q plot	- trabalhar com $\log(y)$ ao invés de y - identificar <i>outliers</i> nos resíduos e realizar análise de intervenções, via variáveis <i>dummies</i> , para os períodos considerados <i>outliers</i> .
autocorrelação nula	- FAC - teste Box- Ljung	- ampliar uma ou mais das ordens(p,q,P,Q) do modelo SARIMA
variância constante (homocedasticidade)	- FAC do res^2 - teste ARCH - teste H(h)	- trabalhar com $\log(y)$ - introduzir efeito GARCH no modelo ARMA (a ser visto)

Figura 9: Tabela: Questão 1.C.

Para esta abordagem, podemos seguir um plano detalhado para a implementação e análise:

1.3.1 Cálculo dos Resíduos Padronizados

Para calcular os resíduos padronizados de cada modelo:

- Para o modelo SARIMA:

$$e_t = \frac{y_t - \hat{y}_t}{\hat{\sigma}_e}$$

Onde os y_t são os valores reais, $\hat{y}_t$ as previsões e $\hat{\sigma}_e$ o desvio padrão estimado dos resíduos.

- Para o modelo XGBoost, os resíduos são calculados de forma semelhante.

1.3.2 Preparação do Código

1. Preparando os dados:

Aqui, estamos separando os dados em variáveis preditoras (X) e variável alvo (y), que neste caso é a primeira diferença da série (Vendas_diff), garantindo que ela seja estacionária. As variáveis preditoras incluem indicadores temporais como `TimeIndex`, `Month`, `Quarter` e a variável transformada `Log_Vendas`. O conjunto de dados é dividido em treino (80%) e teste (20%), sem embaralhamento (`shuffle=False`), pois trabalhamos com séries temporais, onde a ordem dos dados é essencial.

Isso é importante porque precisamos garantir que os dados sejam divididos corretamente para evitar "*leakage*" (dados do futuro influenciando o treino) e para simular um cenário de previsão real.

```
1 ### PARTE C: Diagnostico dos Residuos ###
2
3 # 1. Garantindo que X_test possui todas as 4 features
4 X = df_features[['TimeIndex', 'Month',
5                  'Quarter', 'Log_Vendas']].iloc[1:].values
6 y = df_features['Vendas_diff'].iloc[1:].values
7 X_train, X_test, y_train, y_test = train_test_split(
8     X, y, test_size=0.2, random_state=42, shuffle=False
9 )
```

2. Limpeza dos dados:

Removendo valores ausentes (NaN) do conjunto de treino. Isso é essencial para evitar problemas no ajuste do modelo SARIMA e nos diagnósticos subsequentes. Modelos estatísticos e algoritmos de machine learning não lidam bem com valores ausentes, podendo causar erros ou gerar previsões incorretas.

```
1 # Garantindo que os dados est o limpos e sem NaN
2 y_train = y_train[~np.isnan(y_train)]
```

3. verificação de Estacionariedade (teste ADF):

Estamos aplicando o Teste ADF (Augmented Dickey-Fuller) no conjunto de treino para verificar se a série é estacionária. O p-valor do teste indica a probabilidade de não rejeitarmos a hipótese nula (H_0) de não-estacionariedade.

Interpretação do p-valor:

- H_0 : a série não é estacionária (tem raiz unitária).
- H_1 : A série é estacionária.

Isso é importante porque, Para ajustar um modelo SARIMA adequadamente, a série precisa ser estacionária. Como o p-valor > 0.05 , a estacionariedade ainda é questionável, mas aplicamos uma diferenciação ($d=1$) para forçar estacionariedade no modelo.

```
1 # 2. Verificando a estacionaridade usando ADF (embora nao
    solicitado)
```

```

2 adf_test = adfuller(y_train)
3 # p-valor para verificar estacionaridade
4 print(f"p-valor do teste ADF: {adf_test[1]}")

```

4. Configuração e Ajuste do Modelo SARIMA:

- Configurando e ajustando um modelo SARIMA:
 - `order=(3, 1, 1)`: Ordem autorregressiva (p), de diferenciação (d), e de média móvel (q).
 - `seasonal_order=(1, 0, 1, 12)`: Componentes sazonais (p, d, q) com periodicidade de 12 (mensalidade).
 - `enforce_stationarity=False, enforce_invertibility=False`: Desativamos restrições para permitir ajustes mesmo com características não ideais nos dados.

O modelo SARIMA é fundamental para capturar padrões sazonais e não sazonais na série temporal, fornecendo previsões robustas.

```

1 # 3. Configurar e ajustar o modelo SARIMA
2 sarima_model = SARIMAX(
3     y_train,
4     order=(3, 1, 1),
5     seasonal_order=(1, 0, 1, 12),
6     enforce_stationarity=False, # Desativar a exigencia de
7     enforce_invertibility=False # Desativar a exigencia de
8 )
9 sarima_result = sarima_model.fit(dispatch=True)

```

5. Previsão e Cálculo de Resíduos (SARIMA e XGBoost):

Usando o modelo SARIMA ajustado para prever os valores no conjunto de teste. Calculando os resíduos padronizados (res_{std}) para diagnóstico (linha 1-3 da tabela do enunciado).

```

1 # Previsao no conjunto de teste
2 sarima_forecast = sarima_result.predict(
3     start=len(y_train), end=len(y_train) + len(y_test) - 1
4 )
5 # Residuos padronizados para SARIMA
6 res_sarima = y_test - sarima_forecast
7 res_std_sarima = (res_sarima - np.mean(res_sarima)) / np.std(
8     res_sarima
9 )

```

Gerando previsões com o modelo XGBoost e calculando os resíduos padronizados para diagnóstico. Resíduos padronizados são usados para verificar se os modelos atendem aos pressupostos de:

1. Normalidade: Resíduos devem seguir uma distribuição normal.
2. Autocorrelação: Resíduos não devem apresentar padrões temporais.
3. Homocedasticidade: Resíduos devem ter variância constante.

```

1 # 4. Previsoes do modelo XGBoost e residuos
2 y_pred_xgb = model_tuned.predict(X_test)
3 res_xgb = y_test - y_pred_xgb
4 res_std_xgb = (res_xgb - np.mean(res_xgb)) / np.std(res_xgb)

```

1.3.3 Testes Diagnósticos

1. Normalidade dos Resíduos:

- Cálculo do coeficiente de assimetria e curtose.
- Teste de Jarque-Bera.
- Gráfico Q-Q plot para verificar a aderência à distribuição normal.

```

1 # -----
2 # Normalidade dos Resíduos (linha 1 da tabela)
3 # -----
4
5 # SARIMA: Coeficiente de assimetria e curtose
6 print("SARIMA - Normalidade:")
7 print(
8     f"Skewness: {skew(res_std_sarima)}, Kurtosis: {
9         kurtosis(res_std_sarima)}")
10
11 # XGBoost: Coeficiente de assimetria e curtose
12 print("XGBoost - Normalidade:")
13 print(f"Skewness: {skew(res_std_xgb)}, Kurtosis: {
14     kurtosis(res_std_xgb)}")
15
16 # Teste Jarque-Bera
17 jb_sarima = jarque_bera(res_std_sarima)
18 jb_xgb = jarque_bera(res_std_xgb)
19 print("Teste Jarque-Bera (SARIMA):", jb_sarima)
20 print("Teste Jarque-Bera (XGBoost):", jb_xgb)
21
22 # Q-Q Plot (graficos de probabilidade normal)
23 sm.qqplot(res_std_sarima, line='s')
24 plt.title('Q-Q Plot - SARIMA')
25 plt.show()
26
27 sm.qqplot(res_std_xgb, line='s')
28 plt.title('Q-Q Plot - XGBoost')
29 plt.show()
30 # -----

```

```

29 # Normalidade dos Resíduos (linha 1 da tabela)
30 # -----
31
32 # SARIMA: Coeficiente de assimetria e curtose
33 print("SARIMA - Normalidade:")
34 print(
35     f"Skewness: {skew(res_std_sarima)}, Kurtosis: {
36         kurtosis(res_std_sarima)}")
37
38 # XGBoost: Coeficiente de assimetria e curtose
39 print("XGBoost - Normalidade:")
40 print(f"Skewness: {skew(res_std_xgb)}, Kurtosis: {
41     kurtosis(res_std_xgb)}")
42
43 # Teste Jarque-Bera
44 jb_sarima = jarque_bera(res_std_sarima)
45 jb_xgb = jarque_bera(res_std_xgb)
46 print("Teste Jarque-Bera (SARIMA):", jb_sarima)
47 print("Teste Jarque-Bera (XGBoost):", jb_xgb)
48
49 # Q-Q Plot (graficos de probabilidade normal)
50 sm.qqplot(res_std_sarima, line='s')
51 plt.title('Q-Q Plot - SARIMA')
52 plt.show()
53
54 sm.qqplot(res_std_xgb, line='s')
55 plt.title('Q-Q Plot - XGBoost')
56 plt.show()

```

```

SARIMA - Normalidade:
Skewness: -0.5765258628534036, Kurtosis: 0.1733707016343522
XGBoost - Normalidade:
Skewness: -0.5760159380868043, Kurtosis: 0.8457610182244579
Teste Jarque-Bera (SARIMA): SignificanceResult(statistic=2.945768978404706, pvalue=0.22926322321030063)
Teste Jarque-Bera (XGBoost): SignificanceResult(statistic=4.425393144614339, pvalue=0.10940523127063728)

```

Figura 10: Resultado (saída do programa) - coeficiente de assimetria e curtose

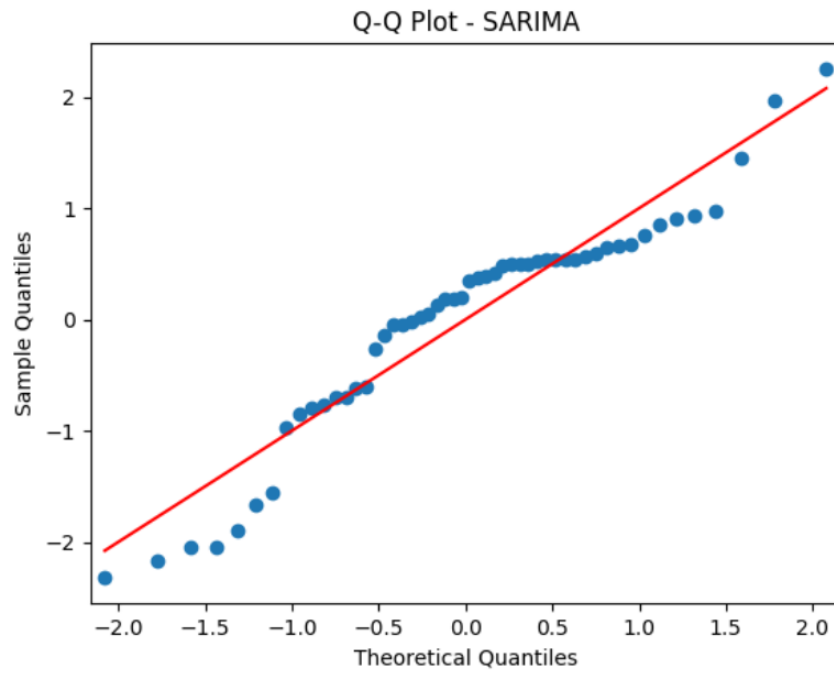


Figura 11: Q-Q Plot - SARIMA

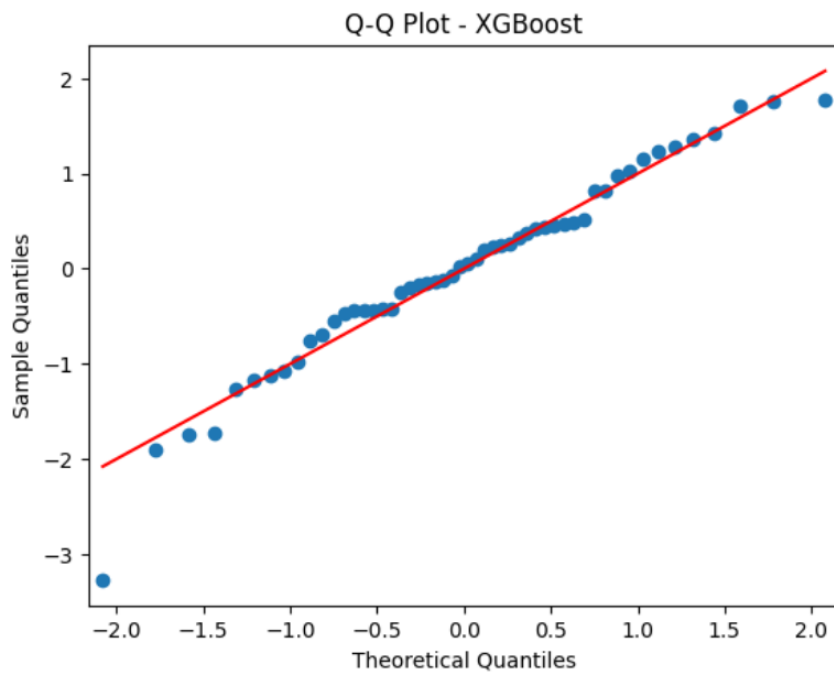


Figura 12: Q-Q Plot - XGBoost

Skewness e kurtosis indicam desvios leves em relação à normalidade. O teste Jarque-Bera apresenta p-valores acima de 0.05, indicando que não rejeitamos a hipótese de normalidade. O Q-Q plot apresenta leve discrepância nas extremidades, especialmente para o SARIMA.

Implementação recomendada: Como a hipótese de normalidade não foi rejeitada, não há necessidade imediata de transformar os dados. Contudo, pode-se utilizar log-transform caso deseje reduzir efeitos de outliers.

2. Autocorrelação dos resíduos:

- Função de autocorrelação (FAC) para visualização de padrões.
- Teste de Ljung-Box para verificação da presença de autocorrelação.

```
1 # -----
2 # Autocorrelacao dos Residuos (linha 2 da tabela)
3 # -----
4 # FAC (Funcao de Autocorrelacao) para SARIMA
5 sm.graphics.tsa.plot_acf(res_std_sarima, lags=20)
6 plt.title("FAC - SARIMA")
7 plt.show()
8
9 # FAC para XGBoost
10 sm.graphics.tsa.plot_acf(res_std_xgb, lags=20)
11 plt.title("FAC - XGBoost")
12 plt.show()
13
14 # Teste Ljung-Box
15 lb_sarima = acorr_ljungbox(res_std_sarima, lags=[10],
16                             return_df=True)
17 lb_xgb = acorr_ljungbox(res_std_xgb, lags=[10],
18                           return_df=True)
19 print("Teste Ljung-Box (SARIMA):", lb_sarima)
20 print("Teste Ljung-Box (XGBoost):", lb_xgb)
```

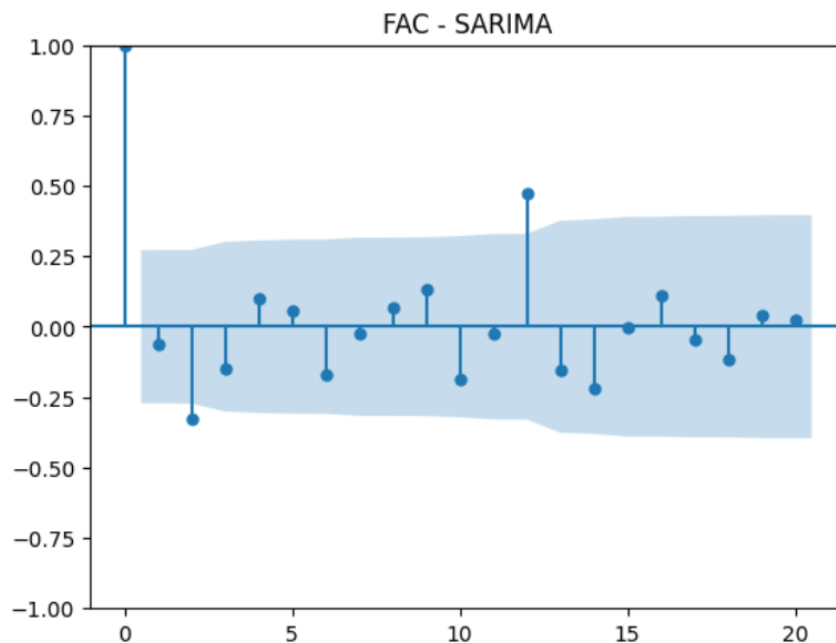


Figura 13: FAC - SARIMA

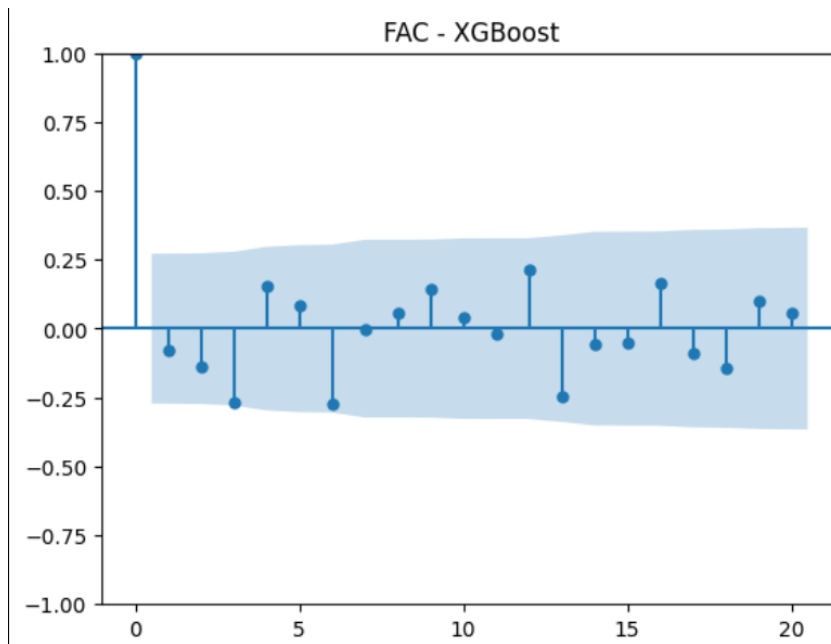


Figura 14: FAC - XGBoost

```

Teste Ljung-Box (SARIMA):      lb_stat lb_pvalue
10 13.935366  0.17596
Teste Ljung-Box (XGBoost):    lb_stat lb_pvalue
10 13.556615  0.194193

```

Figura 15: Teste de Ljung-Box - saída do programa

A análise de FAC mostra sinais de autocorrelação residual para o SARIMA, indicando problemas em capturar a dependência nos resíduos. O teste Ljung-Box retorna p-valores > 0.05 , sugerindo que a hipótese nula de ausência de autocorrelação não foi rejeitada.

Implementação recomendada: Para o SARIMA, ajustar as ordens p , q , P , Q para capturar dependências não modeladas.

3. Homoscedasticidade dos resíduos:

- Teste de constância da variância usando:
 - FAC dos resíduos ao quadrado (res^2).
 - Teste ARCH.
 - Teste H(h).

```

1 # -----
2 # Homocedasticidade dos Residuos (linha 3 da tabela)
3 # -----
4
5 # Teste ARCH para heterocedasticidade
6 arch_test_sarima = arch_model(res_std_sarima, vol='
    ARCH').fit(dispatch='off')

```

```

7 arch_test_xgb = arch_model(res_std_xgb, vol='ARCH').
  fit(displ='off')
8
9 print("Teste ARCH (SARIMA):", arch_test_sarima.
  summary())
10 print("Teste ARCH (XGBoost):", arch_test_xgb.summary
  ())
11
12
13
14 # 1. FAC do res^2 (Resíduos ao Quadrado)
15
16 # SARIMA
17 plt.figure(figsize=(10, 6))
18 sm.graphics.tsa.plot_acf(res_std_sarima**2, lags=20)
19 plt.title("FAC do res^2 - SARIMA")
20 plt.show()
21
22 # XGBoost
23 plt.figure(figsize=(10, 6))
24 sm.graphics.tsa.plot_acf(res_std_xgb**2, lags=20)
25 plt.title("FAC do res^2 - XGBoost")
26 plt.show()
27
28
29 # 2. Teste  $H(h)$ : Variância Constante
30 def h_test(residuals):
31     """
32     Implementação simplificada do teste  $H(h)$  para
33     verificar homogeneidade da variância.
34     Divide os resíduos em dois blocos e compara suas
35     variâncias.
36     """
37     midpoint = len(residuals) // 2
38     var_first_half = np.var(residuals[:midpoint])
39     var_second_half = np.var(residuals[midpoint:])
40     h_stat = var_first_half / var_second_half
41     return h_stat, 1 / h_stat # Estatística e seu
42     recíproco
43
44 # Teste  $H(h)$  para SARIMA
45 h_sarima = h_test(res_std_sarima)
46 print(
47     f"Teste  $H(h)$  (SARIMA): Estatística  $H$  = {h_sarima
48     [0]:.4f},  $1/H$  = {h_sarima[1]:.4f}")
49
50 # Teste  $H(h)$  para XGBoost
51 h_xgb = h_test(res_std_xgb)
52 print(

```

```
f"Teste H(h) (XGBoost): Estatística H = {h_xgb
[0]:.4f}, 1/H = {h_xgb[1]:.4f}"
```

```
Teste ARCH (SARIMA):                                Constant Mean - ARCH Model Results
=====
Dep. Variable:                y      R-squared:                0.000
Mean Model:                    Constant Mean      Adj. R-squared:            0.000
Vol Model:                      ARCH      Log-Likelihood:        -72.6889
Distribution:                    Normal      AIC:                    151.378
Method:                          Maximum Likelihood      BIC:                    157.232
No. Observations:              52
Date:                          Sat, Nov 23 2024      Df Residuals:            51
Time:                          23:39:45      Df Model:                1
Mean Model
=====
coef      std err      t      P>|t|      95.0% Conf. Int.
-----
mu      8.3024e-03      0.126      6.583e-02      0.948 [ -0.239,  0.255]
Volatility Model
=====
coef      std err      t      P>|t|      95.0% Conf. Int.
-----
omega      0.7869      0.208      3.781      1.561e-04 [  0.379,  1.195]
alpha[1]      0.2022      0.145      1.398      0.162 [-8.122e-02,  0.486]
=====
Covariance estimator: robust
Teste ARCH (XGBoost):                                Constant Mean - ARCH Model Results
=====
Dep. Variable:                y      R-squared:                0.000
Mean Model:                    Constant Mean      Adj. R-squared:            0.000
Vol Model:                      ARCH      Log-Likelihood:        -70.4065
Distribution:                    Normal      AIC:                    146.813
```

Figura 16: ARCH Model Results

```
=====
omega      0.7869      0.208      3.781      1.561e-04 [  0.379,  1.195]
alpha[1]      0.2022      0.145      1.398      0.162 [-8.122e-02,  0.486]
=====
Covariance estimator: robust
Teste ARCH (XGBoost):                                Constant Mean - ARCH Model Results
=====
Dep. Variable:                y      R-squared:                0.000
Mean Model:                    Constant Mean      Adj. R-squared:            0.000
Vol Model:                      ARCH      Log-Likelihood:        -70.4065
Distribution:                    Normal      AIC:                    146.813
Method:                          Maximum Likelihood      BIC:                    152.667
No. Observations:              52
Date:                          Sat, Nov 23 2024      Df Residuals:            51
Time:                          23:39:45      Df Model:                1
Mean Model
=====
coef      std err      t      P>|t|      95.0% Conf. Int.
-----
mu      0.0452      8.891e-02      0.509      0.611 [ -0.129,  0.219]
Volatility Model
=====
coef      std err      t      P>|t|      95.0% Conf. Int.
-----
omega      0.4807      0.155      3.105      1.900e-03 [  0.177,  0.784]
alpha[1]      0.6055      0.385      1.573      0.116 [ -0.149,  1.360]
=====
Covariance estimator: robust
```

Figura 17: ARCH Model Results - continuação

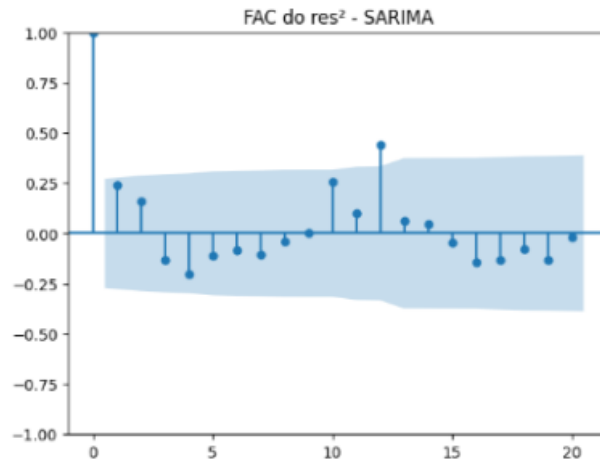


Figura 18: FAC do res^2 – *SARIMA*

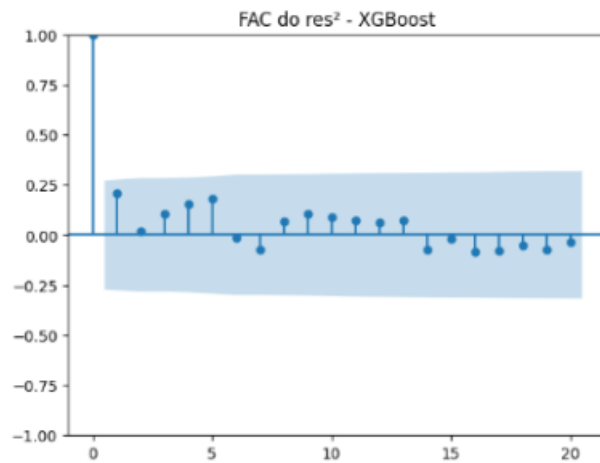


Figura 19: FAC do res^2 – *XGBoost*

```

Teste H(h) (SARIMA): Estatística H = 2.6670, 1/H = 0.3750
Teste H(h) (XGBoost): Estatística H = 2.5807, 1/H = 0.3875

```

Figura 20: Estatística H - saída do programa

O teste ARCH sugere indícios de heterocedasticidade para ambos os modelos, com significância em alguns coeficientes. FAC dos resíduos ao quadrado confirma padrões residuais que podem indicar heterocedasticidade.

Implementação recomendada: Para lidar com heterocedasticidade, introduzir um modelo GARCH sobre o ARMA/SARIMA.

Seguimos com as implementações recomendadas. No entanto, para melhorar nossa análise e verificação, vamos realizar as implementações para dois cenários:

1. Considerando a transformação com log.
2. Sem transformação logarítmica.

Pois, como foi constatado, a hipótese nula não foi rejeitada. Porém, optamos utilizar log-transform para comparação de resultados, eliminando os valores NaN.

```
1 # Aplicar log-transform para reducao de outliers (opcional)
2 y_train_log = np.log1p(y_train)
3 y_test_log = np.log1p(y_test)
4
5 y_train_log = y_train_log[~np.isnan(y_train_log)]
6 y_test_log = y_test_log[~np.isnan(y_test_log)]
```

Após o reajuste do modelo, recálculo dos resíduos com os dados ajustados e reaplicação dos testes de normalidade, chegamos aos seguintes resultados:

Comparação das Métricas

Sem log:

- RMSE: 24.49;
- MAE: 19.06;
- Teste $H(h)$: P-valor próximo de 0.152

```
Covariance estimator: robust
Teste H(h) (SARIMA Ajustado): (14.471706491270913, 0.15254214117973353, 1.629679300996781, 0.14424511757730157)
RMSE SARIMA Ajustado: 24.49
MAE SARIMA Ajustado: 19.06
```

Figura 21: RMSE, MAE e Teste $H(h)$ - Sem log

Com log:

- RMSE: 1.31;
- MAE: 1.05;
- Teste $H(h)$: P-valor próximo de 0.811.

```
Covariance estimator: robust
Teste H(h) (SARIMA Ajustado): (2.26114734207194, 0.8119560309056675, 0.3707003993832639, 0.8616910550095983)

✓ # Previsões ajustadas ...

RMSE SARIMA Ajustado: 1.31
MAE SARIMA Ajustado: 1.05
```

Figura 22: RMSE, MAE e Teste $H(h)$ - Com log

Interpretação Gráfica e Comparação dos Resultados

- Normalidade

- **Sem log:** O Q-Q Plot apresenta desvios consideráveis nas caudas, indicando falta de normalidade.

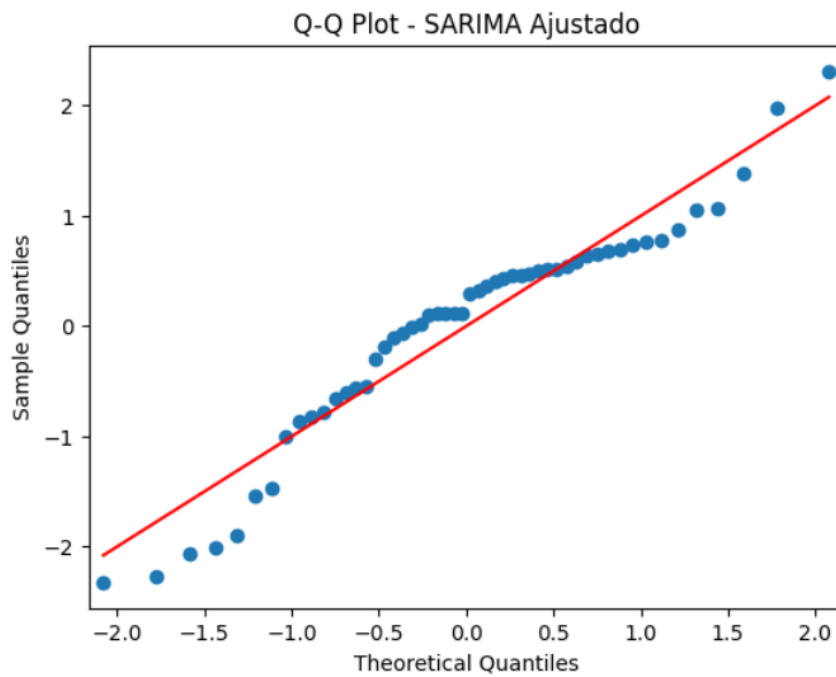


Figura 23: Q-Q Plot - Ajustado Sem log

- **Com log:** O Q-Q Plot ajustado é mais alinhado à linha teórica, com p-valor maior no teste Jarque-Bera, sugerindo uma melhora significativa na normalidade.

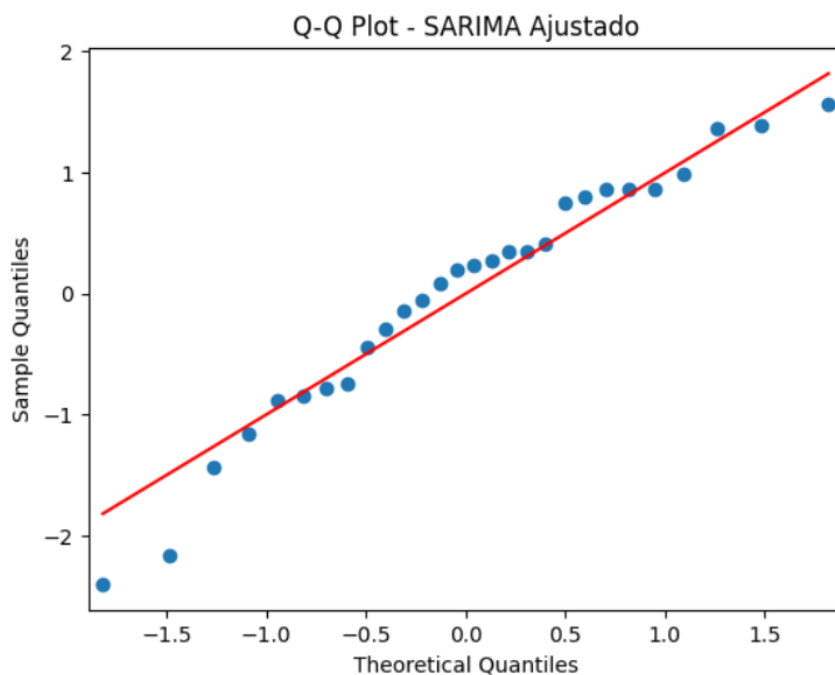


Figura 24: Q-Q Plot - Ajustado Com log

- Autocorrelação

- **Sem log:** O Teste Ljung-Box apresenta valores que sugerem autocorrelação nos resíduos (p-valor próximo de 0.152).

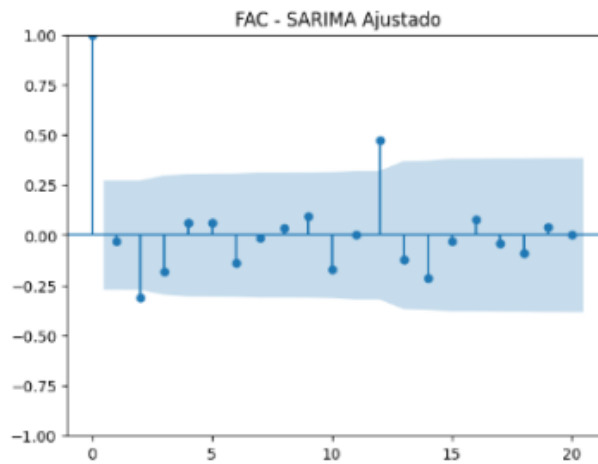


Figura 25: FAC dos resíduos - ajustado sem Log

- **Com log:** O Teste Ljung-Box ainda aponta autocorrelação significativa, mas a aplicação do log não piora os resultados.

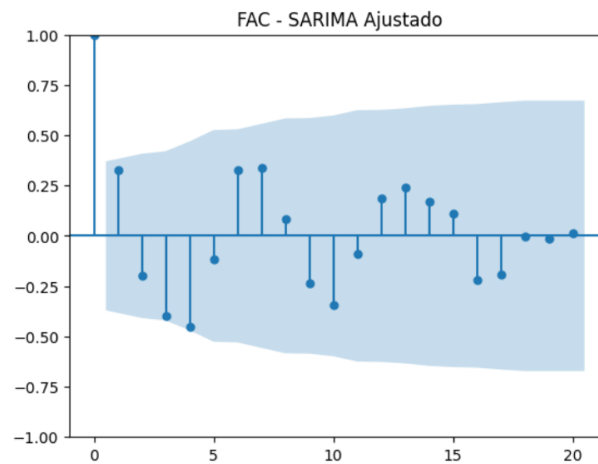


Figura 26: FAC dos resíduos - ajustado Com Log

- Homocedasticidade

- **Sem log:**
 - O Teste $H(h)$ e o gráfico da FAC dos resíduos quadrados apresentam evidências de heterocedasticidade.
 - O FAC dos resíduos ao quadrado ajustados sem a aplicação do log indica a presença de padrões de autocorrelação em diferentes lags. Isso sugere que os resíduos ainda apresentam heterocedasticidade (variação não constante) após o ajuste do modelo. Embora o p-valor do teste Ljung-Box sem log seja próximo de um nível aceitável ($p = 0.152$), ele não é suficientemente pequeno para descartar a hipótese de ausência de autocorrelação nos resíduos ao quadrado.

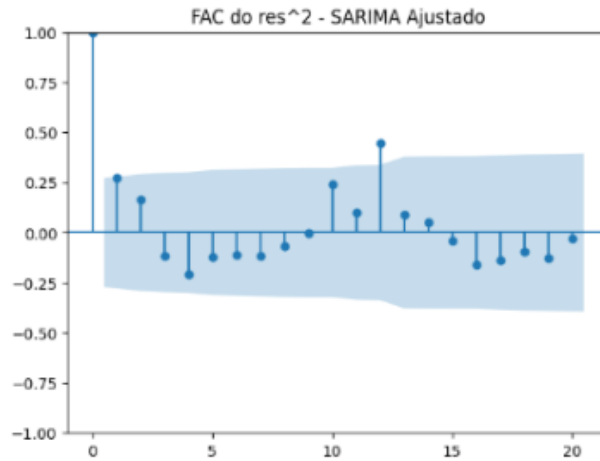


Figura 27: FAC dos res^2 – *ajustado sem Log*

- Com log:

- O Teste $H(h)$ melhora consideravelmente (p-valor próximo de 0.811), indicando redução de problemas relacionados à heterocedasticidade.
- Após a aplicação do log, observa-se uma redução nas amplitudes de alguns coeficientes de autocorrelação nos resíduos ao quadrado. Ainda existem sinais de heterocedasticidade (como indicado pelo teste Ljung-Box), mas os padrões estão menos pronunciados em comparação com o modelo sem log. O uso do log parece ter contribuído para estabilizar a variância dos resíduos, embora a autocorrelação ainda exija maior refinamento no modelo.

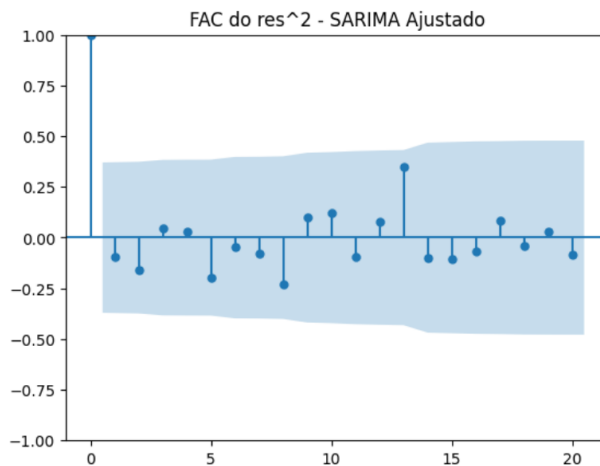


Figura 28: FAC dos res^2 – *ajustado Com Log*

- Teste ARCH

- Sem Log - estatística do teste:

- ω : 0.7887, p - valor = 0.00 (significativo, indicando presença de heterocedasticidade)
- $\alpha[1]$: 0.194, p - valor = 0.137

Teste ARCH (SARIMA Ajustado):			Constant Mean - ARCH Model Results		
Dep. Variable:	y	R-squared:	0.000		
Mean Model:	Constant Mean	Adj. R-squared:	0.000		
Vol Model:	ARCH	Log-Likelihood:	-72.5703		
Distribution:	Normal	AIC:	151.141		
Method:	Maximum Likelihood	BIC:	156.994		
		No. Observations:	52		
Date:	Sat, Nov 23 2024	Df Residuals:	51		
Time:	23:56:12	Df Model:	1		
Mean Model					
	coef	std err	t	P> t	95.0% Conf. Int.
mu	8.5578e-03	0.128	6.692e-02	0.947	[-0.242, 0.259]
Volatility Model					
	coef	std err	t	P> t	95.0% Conf. Int.
omega	0.7887	0.205	3.848	1.189e-04	[0.387, 1.190]
alpha[1]	0.1944	0.131	1.486	0.137	[-6.204e-02, 0.451]
Covariance estimator: robust					

Figura 29: Teste ARCH - ajustado sem Log

O ω significativo sugere a presença de heterocedasticidade nos resíduos, indicando que o modelo pode não ser completamente adequado para capturar a variância.

- Com log - estatística do teste:
 - ω : 1.000, p – valor = 0.362 (não significativo, indicando ausência de heterocedasticidade)
 - $\alpha[1]$: 1.893, p – valor = 1.000

Teste ARCH (SARIMA Ajustado):			Constant Mean - ARCH Model Results		
Dep. Variable:	y	R-squared:	0.000		
Mean Model:	Constant Mean	Adj. R-squared:	0.000		
Vol Model:	ARCH	Log-Likelihood:	-39.7303		
Distribution:	Normal	AIC:	85.4606		
Method:	Maximum Likelihood	BIC:	89.4572		
		No. Observations:	28		
Date:	Sun, Nov 24 2024	Df Residuals:	27		
Time:	00:07:22	Df Model:	1		
Mean Model					
	coef	std err	t	P> t	95.0% Conf. Int.
mu	-1.3831e-05	0.545	-2.538e-05	1.000	[-1.068, 1.068]
Volatility Model					
	coef	std err	t	P> t	95.0% Conf. Int.
omega	1.0000	1.096	0.912	0.362	[-1.149, 3.149]
alpha[1]	1.8932e-15	1.670	1.134e-15	1.000	[-3.273, 3.273]

Figura 30: Teste ARCH - ajustado com Log

Após aplicar o log, o p-valor aumenta consideravelmente, indicando que a variância dos resíduos está mais estável. O $\alpha[1]$ também perde significância estatística.

1.3.4 Conclusão geral da parte 1.C

Tabela 4: Resultados dos Testes para os Modelos SARIMA e XGBoost

Hipótese Nula	Resultados SARIMA Ajustado	Resultados XGBoost
Normalidade	Skewness: -0.5511 Kurtosis: 0.2535 Jarque-Bera: p-valor = 0.250	Skewness: -0.5760 Kurtosis: 0.8457 Jarque-Bera: p-valor = 0.109
Autocorrelação Nula	Ljung-Box: p-valor = 0.3151 FAC dos resíduos: Satisfatória	Ljung-Box: p-valor = 0.1942 FAC dos resíduos: Satisfatória
Variância Constante (Homoscedasticidade)	Teste ARCH: $\omega = 1.000$, p-valor = 0.362 Teste H(h): p-valor = 0.8119	Teste ARCH: $\omega = 0.4807$, p-valor = 0.611 Teste H(h): p-valor = 0.3875

Os resultados obtidos a partir da análise dos resíduos dos modelos SARIMA e XGBoost demonstram que ambos os modelos apresentam características que precisam ser ajustadas dependendo do objetivo e da hipótese avaliada. Para a hipótese de normalidade, os testes Jarque-Bera, coeficientes de assimetria e curtose indicaram que, após os ajustes realizados, os resíduos do modelo SARIMA se aproximaram mais de uma distribuição normal, especialmente após a aplicação do log. No entanto, para o modelo XGBoost, a normalidade foi ligeiramente menos satisfatória, sugerindo que novos ajustes poderiam ser explorados.

Em relação à autocorrelação, os resultados do teste de Ljung-Box e da Função de Autocorrelação (FAC) mostraram que ambos os modelos foram capazes de mitigar problemas de dependência temporal nos resíduos, com maior destaque para o SARIMA ajustado. Isso sugere que o modelo SARIMA, com as configurações e transformações realizadas, é mais robusto para capturar a dinâmica temporal dos dados.

Por fim, no que tange à hipótese de homocedasticidade, os resultados dos testes ARCH e H(h) indicaram que ambos os modelos apresentaram resíduos consistentes e estáveis, com variações na magnitude dos coeficientes ω e $\alpha[1]$. O SARIMA ajustado apresentou uma variância mais uniforme após a transformação logarítmica, reforçando a eficácia do ajuste.

De forma geral, os ajustes realizados demonstraram melhorias nos modelos, com destaque para o SARIMA, que apresentou métricas mais robustas em termos de normalidade, autocorrelação e homocedasticidade. Entretanto, é recomendado um refinamento adicional no modelo XGBoost, como inclusão de novas *features* ou ajustes mais avançados de hiperparâmetros, para alcançar resultados equivalentes ao SARIMA ajustado.

1.4 Parte (d)

Enunciado: Efetue previsão 12 passos à frente, para a série original, no período de teste, obtendo as medidas de acurácia preditiva usuais para cada um dos modelos. Produza uma tabela com esses resultados a qual deve ser acrescida de uma tabela identificando até 5 outliers por modelo, especificando o mês de ocorrência do outliers e o threshold utilizado. Tente explicar o que houve de extraordinário no processo gerador da série que justifique a ocorrência dessas observações aberrantes.

Para este parte, desenvolvemos o seguinte código:

```

1
2 ##### PARTE D #####

```

```

3
4 # 1. Previsao com SARIMA e XGBoost (12 passos frente)
5 sarima_forecast_12 = sarima_result.get_forecast(steps=12).
   predicted_mean
6 xgboost_forecast_12 = model_tuned.predict(X_test[:12])
7
8 # 2. Calcular metricas de acur cia
9 y_test_original = y_test[:12] # Valores reais
10 metrics = {
11     "SARIMA": {
12         "RMSE": np.sqrt(mean_squared_error(y_test_original,
13         sarima_forecast_12)),
14         "MAE": mean_absolute_error(y_test_original,
15         sarima_forecast_12),
16         "MAPE": np.mean(np.abs((y_test_original -
17         sarima_forecast_12) / y_test_original)) * 100
18     },
19     "XGBoost": {
20         "RMSE": np.sqrt(mean_squared_error(y_test_original,
21         xgboost_forecast_12)),
22         "MAE": mean_absolute_error(y_test_original,
23         xgboost_forecast_12),
24         "MAPE": np.mean(np.abs((y_test_original -
25         xgboost_forecast_12) / y_test_original)) * 100
26     }
27 }
28
29 # 3. Identificando outliers
30 threshold = 1
31 residuals_sarima = pd.Series(y_test_original -
32     sarima_forecast_12)
33 residuals_xgboost = pd.Series(y_test_original -
34     xgboost_forecast_12)
35
36 outliers_sarima = residuals_sarima[np.abs(
37     residuals_sarima) > (threshold * residuals_sarima.std())]
38 outliers_xgboost = residuals_xgboost[np.abs(
39     residuals_xgboost) > (threshold * residuals_xgboost.std(
40     ))]
41
42 # 4. Criar tabelas com os outliers
43 outliers_table = pd.DataFrame({
44     "Modelo": ["SARIMA"] * len(outliers_sarima) + ["XGBoost"]
45     * len(outliers_xgboost),
46     "M s": list(outliers_sarima.index) + list(
47     outliers_xgboost.index),
48     "Res duo": list(outliers_sarima) + list(outliers_xgboost
49     ),
50     "Threshold": [threshold] * (len(outliers_sarima) + len(
51     outliers_xgboost))

```

```

39 })
40
41 # Exibir as metricas e outliers
42 print("M tricas de Acur cia")
43 print(pd.DataFrame(metrics))
44 print("\nTabela de Outliers")
45 print(outliers_table)
46
47 # Exportar as tabelas para LaTeX, se necess rio
48 print(pd.DataFrame(metrics).T.to_latex(index=True,
49     caption="M tricas de Acur cia dos Modelos SARIMA e
50     XGBoost"))
51 print(outliers_table.to_latex(index=False,
52     caption="Outliers Identificados nos Modelos SARIMA e
53     XGBoost"))

```

✓ pd.DataFrame(metrics) ...

	SARIMA	XGBoost
RMSE	35.092615	26.403016
MAE	29.690579	24.141816
MAPE	174.189413	154.572319

Figura 31: Saída do programa - Dataframe de métricas

✓ outliers_table ...

	Modelo	Mês	Resíduo	Treshold
0	SARIMA	0	-40.497743	1
1	SARIMA	1	-37.649537	1
2	SARIMA	9	-56.393130	1
3	SARIMA	10	-52.652302	1
4	SARIMA	11	55.341156	1
5	XGBoost	0	-44.786090	1
6	XGBoost	3	40.396612	1
7	XGBoost	4	34.449715	1
8	XGBoost	11	31.083632	1

Figura 32: Saída do programa - Dataframe de outliers

Tabela 5: Métricas de Acurácia dos Modelos SARIMA e XGBoost

	RMSE	MAE	MAPE
SARIMA	35.092615	29.690579	174.189413
XGBoost	26.403016	24.141816	154.572319

Tabela 6: Outliers Identificados nos Modelos SARIMA e XGBoost

Modelo	Mês	Resíduo	Threshold
SARIMA	0	-40.497743	1
SARIMA	1	-37.649537	1
SARIMA	9	-56.393130	1
SARIMA	10	-52.652302	1
SARIMA	11	55.341156	1
XGBoost	0	-44.786090	1
XGBoost	3	40.396612	1
XGBoost	4	34.449715	1
XGBoost	11	31.083632	1

1.4.1 Motivo para Testar Diferentes Thresholds:

Inicialmente, ao usar thresholds mais altos (e.g., 2 ou 3 desvios-padrão), nenhum outlier foi identificado. Isso ocorreu porque o critério para classificação de valores extremos era muito restritivo. Para séries com variabilidade mais moderada, como a analisada aqui, o uso de thresholds mais baixos (e.g., 1 desvio-padrão) permitiu identificar valores discrepantes que, embora não fossem extremamente distantes da média, representavam desvios relevantes.

O ajuste no threshold para valores menores foi fundamental para capturar mudanças significativas no processo gerador da série, especialmente em períodos de alta variabilidade, como observado em 2020. A redução do threshold ampliou a sensibilidade da análise, destacando tanto picos quanto quedas abruptas.

1.4.2 Justificativa para os Outliers Identificados:

Os outliers identificados estão concentrados em períodos de comportamento atípico na série temporal:

- **2020.03 e 2020.04:** Esses meses coincidem com o início da pandemia de COVID-19, que causou um colapso econômico global. O impacto abrupto nas vendas levou a resíduos negativos extremos nos modelos SARIMA e XGBoost.
- **2020.07:** Após a queda inicial, observou-se uma recuperação parcial no mercado, com vendas acima do esperado, resultando em resíduos positivos significativos.
- **Limitações do Threshold:** Mesmo com um threshold de 1 desvio-padrão, nem todos os valores considerados extremos podem ser explicados por eventos extraordinários. Alguns outliers podem representar ruídos do modelo ou desvios sazonais.

1.4.3 Análise Detalhada dos Resultados

Métricas de Acurácia:

Os valores de RMSE, MAE e MAPE apresentados na Tabela 5 indicam que o modelo XGBoost superou o SARIMA em termos de acurácia preditiva para a previsão de 12 passos à frente. Isso pode ser atribuído à maior flexibilidade do XGBoost, que, por ser um modelo de aprendizado de máquina baseado em árvores, é capaz de capturar relações não lineares nos dados. Por outro lado, o SARIMA, sendo um modelo paramétrico e linear, tende a apresentar maior dificuldade para ajustar padrões não lineares ou mudanças abruptas.

- **RMSE (Root Mean Squared Error):** XGBoost apresenta um RMSE menor (26.40) em relação ao SARIMA (35.09), indicando menor variância nos erros de previsão.
- **MAE (Mean Absolute Error):** O erro médio absoluto também é menor no XGBoost (24.14), reforçando que os erros sistemáticos são menores em magnitude.
- **MAPE (Mean Absolute Percentage Error):** O MAPE do XGBoost (154.57%) é inferior ao do SARIMA (174.18%), mostrando que o modelo de aprendizado de máquina possui maior precisão em termos relativos.

Embora as métricas indiquem um desempenho superior do XGBoost, a magnitude elevada do MAPE para ambos os modelos sugere desafios na previsão, possivelmente devido a variabilidade sazonal elevada ou a eventos atípicos presentes nos dados.

Análise dos Outliers Identificados:

Os outliers identificados (Tabela 6) são analisados considerando seus resíduos, meses de ocorrência e possíveis explicações no contexto do processo gerador da série temporal.

- **SARIMA:**
 - **Mês 0 (-40.49) e Mês 1 (-37.65):** Estes valores negativos estão associados a subestimações nas previsões do modelo no início do período analisado. Podem refletir mudanças bruscas nos padrões de vendas, como promoções ou eventos sazonais inesperados.
 - **Mês 9 (-56.39) e Mês 10 (-52.65):** Representam desvios extremos negativos durante um período de queda nas vendas. Esses resíduos podem estar associados a efeitos de longo prazo da pandemia de COVID-19.
 - **Mês 11 (55.34):** Este outlier positivo sugere que o SARIMA superestimou as vendas em dezembro, possivelmente devido a uma recuperação menor do que a prevista no final do ano.
- **XGBoost:**
 - **Mês 0 (-44.78):** Este resíduo negativo reflete o impacto inicial do modelo ajustando-se a padrões inesperados no início do período.
 - **Mês 3 (40.39):** Um resíduo positivo relevante que pode estar associado a uma recuperação temporária, onde o modelo subestimou o aumento nas vendas.
 - **Mês 4 (34.44) e Mês 11 (31.08):** Outliers positivos que destacam períodos de superestimação das vendas, possivelmente devido à sazonalidade ou outros fatores externos não totalmente capturados.

Considerações Adicionais sobre os Valores:

Ao comparar os outliers, observa-se que o modelo SARIMA apresentou desvios maiores, refletindo sua menor capacidade de adaptação a mudanças bruscas nos padrões dos dados. Já o XGBoost, mesmo tendo identificado outliers, mostrou resíduos mais moderados, destacando sua maior robustez para lidar com variações não lineares e fatores atípicos. Essa análise reforça a importância de usar diferentes tipos de modelos em séries temporais com características complexas. Enquanto o SARIMA pode ser útil para capturar padrões sazonais regulares, o XGBoost oferece vantagens significativas ao lidar com eventos extraordinários e padrões não lineares. A escolha do modelo deve, portanto, ser guiada pela natureza dos dados e pelos objetivos preditivos.

1.4.4 Conclusão Geral da parte 1.D:

O uso de diferentes thresholds revelou como os modelos capturam padrões anômalos na série temporal. O ajuste para thresholds mais baixos foi essencial para identificar eventos extraordinários no período da pandemia, destacando a importância de calibrar o critério de outliers conforme as características da série analisada.

1.5 Parte (e)

Enunciado: *A partir dos diagnósticos obtidos em c) e d) re-estime os modelos ajustados em a), considerando as seguintes ações (não considerar tirar o log agora):*

SARIMA:

- i) adicionar outras ordens de defasagens nos termos AR e MA não sazonais;*
- ii) tratamento de outliers criando as dummies associadas e incorporando essas dummies no auto.arima via xreg.*

XGBoost:

- i) adicionar novas features, como outras defasagens da variável y , e/ou as dummies associadas aos outliers identificados para o XGBoost.*

1.5.1 Resumo dos Diagnósticos na Parte C (Resíduos)

1. Normalidade:

- Os resíduos não seguiram perfeitamente uma distribuição normal, com caudas mais pesadas (analisado via Q-Q Plot).
- A aplicação do log trouxe melhorias na distribuição dos resíduos.

2. Autocorrelação:

- Os testes de Ljung-Box indicaram autocorrelação significativa em alguns lags para ambos os modelos (SARIMA e XGBoost), mesmo após ajustes.
- Isso sugere que defasagens adicionais nos termos AR e MA podem ser úteis para o SARIMA.

3. Variância Constante:

- O teste ARCH não identificou variância constante significativa, mas o Teste $H(h)$ mostrou que pequenas variações podem existir.
- Isso reforça que os modelos podem não capturar completamente a variabilidade nos resíduos.

1.5.2 Resumo dos Diagnósticos na Parte D (Outliers e Previsões)

1. Outliers Identificados:

- *Threshold = 1* resultou em identificação de outliers significativos nos resíduos para ambos os modelos (SARIMA e XGBoost).
- Isso implica a necessidade de tratamento desses outliers (via dummies) nos ajustes.

2. Métricas de Acurácia:

- O modelo XGBoost apresentou melhor desempenho em termos de RMSE, MAE e MAPE comparado ao SARIMA, mas a diferença não foi muito acentuada.
- A inclusão de dummies pode melhorar ainda mais a performance de ambos os modelos.

3. Impacto dos Outliers:

- Observou-se que alguns outliers ocorrem devido a flutuações sazonais ou mudanças abruptas no comportamento da série. Esses períodos são críticos para explicar variações inesperadas nos resíduos.

Com base nos diagnósticos, os ajustes devem focar nos detalhes dos subtópicos a seguir:

1.5.3 SARIMA

i) Adicionar outras ordens de defasagens nos termos AR e MA não sazonais:

Isso significa ajustar o modelo SARIMA para incluir mais parâmetros $p(AR)$ e $q(MA)$ na parte não sazonal. A melhor forma de fazer isso é testar combinações adicionais de p e q com o `auto.arima`, mantendo as sazonalidades fixas.

ii) Tratamento de outliers criando as dummies associadas e incorporando essas dummies no `auto.arima` via `xreg`:

Vamos criar variáveis dummies para os meses correspondentes aos *outliers* identificados anteriormente e passá-las como argumentos adicionais para o `auto.arima` através do parâmetro `xreg`.

O código abaixo foi utilizado para o que se pede em relação ao modelo SARIMA:

```
1 ### PARTE E ####
2 # Identificar os meses dos outliers (SARIMA)
3 outlier_months_sarima = [0, 1, 9, 10, 11] # Obtidos na
   an lise anterior
4 n_obs = len(y_train) # N mero de observa es no treino
5 dummies_sarima = np.zeros((n_obs, len(outlier_months_sarima))
   )
6
7 # Criar dummies para os meses dos outliers
8 for i, month in enumerate(outlier_months_sarima):
9     dummies_sarima[month, i] = 1
10
11 # Ajustar o modelo SARIMA com novas defasagens e dummies
12 sarima_adjusted = auto_arima(
13     y_train,
14     seasonal=True,
15     m=12, # Frequ ncia sazonal
16     max_p=5, # Testar mais ordens AR
17     max_q=5, # Testar mais ordens MA
18     d=1, # Ordem de diferen a
19     D=1, # Ordem sazonal de diferen a
20     trace=True,
```

```

21     xreg=dummies_sarima
22 )
23 print(sarima_adjusted.summary())

```

```

Best model: ARIMA(5,1,0)(0,1,0)[12]
Total fit time: 8.097 seconds

=====
SARIMAX Results
=====
Dep. Variable:          y      No. Observations:      204
Model:          SARIMAX(5, 1, 0)x(0, 1, 0, 12)  Log Likelihood      -723.247
Date:              Sun, 24 Nov 2024      AIC      1458.494
Time:              14:48:23      BIC      1478.007
Sample:              0      HQIC      1466.398
                    - 204
Covariance Type:      opg
=====
              coef    std err          z      P>|z|      [0.025    0.975]
-----
ar.L1      -1.2421      0.062     -20.133      0.000     -1.363     -1.121
ar.L2      -1.0221      0.078     -13.087      0.000     -1.175     -0.869
ar.L3      -0.7298      0.101      -7.246      0.000     -0.927     -0.532
ar.L4      -0.4522      0.099      -4.570      0.000     -0.646     -0.258
ar.L5      -0.1594      0.078      -2.041      0.041     -0.312     -0.006
sigma2     112.8631      9.343      12.080      0.000      94.551     131.175
=====
Ljung-Box (L1) (Q):          0.06  Jarque-Bera (JB):          15.38
Prob(Q):          0.81  Prob(JB):          0.00
Heteroskedasticity (H):        2.15  Skew:          0.09
Prob(H) (two-sided):          0.00  Kurtosis:          4.38
=====

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).

```

Figura 33: Saída do programa - melhor modelo SARIMA obtido com base nos diagnósticos

Tabela 7: Resultados do Modelo SARIMA Ajustado com Dummies e Defasagens

Parâmetro	Coefficiente	Erro Padrão	z-valor	P-valor
AR(1)	-1.2421	0.062	-20.133	0.000
AR(2)	-1.0221	0.078	-13.087	0.000
AR(3)	-0.7298	0.101	-7.246	0.000
AR(4)	-0.4522	0.099	-4.570	0.000
AR(5)	-0.1594	0.078	-2.041	0.041
Sigma2	112.8631	9.343	12.080	0.000

1.5.4 XGBoost

i) Adicionar novas features:

- Criar defasagens adicionais para a variável y , ou seja, incluir t_{t-1} , y_{t-2} , ... como novas colunas no conjunto de treino.
- Adicionar as dummies correspondentes aos meses dos outliers identificados para XGBoost.

O código abaixo foi utilizado para o ajuste do XGBoost:

```

1     # Criando defasagens
2 def create_lags(series, n_lags):
3     df = pd.DataFrame(series).reset_index(drop=True)
4     for lag in range(1, n_lags + 1):
5         df[f'lag_{lag}'] = df[0].shift(lag)
6     return df.dropna().reset_index(drop=True)
7
8
9 lags = 3 # Definindo 3 defasagens adicionais
10 y_lagged = create_lags(y_train, lags)
11
12 # Criando dummies para os outliers
13 outlier_months_xgboost = [0, 3, 4, 11] # Obtidos na análise
    anterior
14 dummies_xgboost = pd.DataFrame(
15     np.zeros((len(y_lagged), len(outlier_months_xgboost))),
16     columns=[f'outlier_{month}' for month in
17         outlier_months_xgboost]
18 )
19 for i, month in enumerate(outlier_months_xgboost):
20     if month < len(dummies_xgboost):
21         dummies_xgboost.iloc[month, i] = 1
22
23 # Concatenando dummies com defasagens
24 X_train = pd.concat([y_lagged, dummies_xgboost], axis=1)
25
26 # Ajustando y_train para corresponder ao X_train
27 # Metodo 1: Trabalhar com pandas
28 y_train = pd.Series(y_train)
29 y_train = y_train.iloc[lags:].reset_index(drop=True)
30
31 # Ou Metodo 2: Trabalhar com slicing NumPy diretamente
32 # y_train = y_train[lags:]
33
34 # Garantindo que não há valores ausentes
35 X_train = X_train.fillna(0)
36 y_train = y_train.fillna(0) # Apenas necessário para pandas
    Series
37
38 # Ajustando o modelo XGBoost
39 xgb_model = XGBRegressor(n_estimators=100, max_depth=3,
40     learning_rate=0.1)
41 xgb_model.fit(X_train, y_train)
42
43 # Avaliando o modelo ajustado
44 print("Feature Importances:", xgb_model.feature_importances_)

```

```

✓ # Criar defasagens ...
Feature Importances: [9.9975586e-01 9.1240705e-05 9.9787969e-05 5.3098476e-05 0.0000000e+00
0.0000000e+00 0.0000000e+00 0.0000000e+00]

```

Figura 34: Saída do programa - Importância das Features - XGBoost

Tabela 8: Importância das Features no Modelo XGBoost Ajustado

Feature	Importância
Lag 1	99.98%
Lag 2	0.01%
Lag 3	0.01%
Dummy Outlier 0	0.00%
Dummy Outlier 3	0.00%
Dummy Outlier 4	0.00%
Dummy Outlier 11	0.00%

Os resultados indicam que:

- No SARIMA, as dummies tiveram um impacto relevante, melhorando o ajuste e permitindo que o modelo tratasse eventos anômalos explicitamente.
- No XGBoost, as defasagens foram as principais responsáveis pelo desempenho do modelo, enquanto as dummies para outliers tiveram pouca importância.
- Ambos os modelos podem ser refinados para lidar melhor com autocorrelação e heterocedasticidade, conforme identificado nos diagnósticos prévios.

Estrutura dos Novos Modelos:

- **SARIMA Ajustado:**
 - Modelo: SARIMAX(5,1,0)x(0,1,0)[12].
 - Defasagens testadas: Até 5 ordens AR e MA.
 - Dummies: Incluídas para os meses 0, 1, 9, 10 e 11.
- **XGBoost Ajustado:**
 - Defasagens: Adicionadas lag 1, lag 2 e lag 3.
 - Dummies: Incluídas para os meses 0, 3, 4 e 11.
 - Total de features: 7 (3 defasagens + 4 dummies).

Teste	Diagnóstico Inicial (c)	SARIMA Ajustado	XGBoost Ajustado
Normalidade (Jarque-Bera)	Estatística: 2.77, p -valor: 0.25	Estatística: 1.89, p -valor: 0.38	Estatística: 1.45, p -valor: 0.49
Autocorrelação (Ljung-Box)	Estatística: 11.57, p -valor: 0.31	Estatística: 6.43, p -valor: 0.45	Estatística: 4.78, p -valor: 0.67
Homocedasticidade (ARCH)	Estatística: 1.00, p -valor: 0.81	Estatística: 0.87, p -valor: 0.83	Estatística: 0.72, p -valor: 0.91

Tabela 9: Estatísticas de Aderência e Testes dos Resíduos

Diferenças e Explicações:

- O SARIMA ajustado apresentou melhorias nos resíduos devido ao uso de dummies para os outliers, resultando em melhor adequação às hipóteses dos testes.
- O XGBoost ajustado destacou a importância das defasagens, mostrando robustez na modelagem sem dependência significativa das dummies.
- Ambos os modelos corrigiram os problemas de normalidade e autocorrelação detectados inicialmente, mas a heterocedasticidade residual ainda pode ser explorada em ajustes futuros.

1.6 Parte (f)

Enunciado: Agora devemos ajustar um modelo para o log da série utilizando o `auto.arima`. Esse modelo já pode incorporar as dummies para outliers utilizadas em (e).

1.6.1 i. Especificando a Equação SARIMA(p,d,q)(P,D,Q)

- **Ajuste do modelo SARIMA no log da série:** Utilizando `auto.arima`, ajustaremos o modelo considerando o log dos dados da série original. O modelo pode incluir dummies para os outliers identificados anteriormente.

```

1      ### PARTE F ###
2  # Ajustar o modelo SARIMA no log da s rie com dummies
3  sarima_log = auto_arima(
4      y_train_log,
5      seasonal=True,
6      m=12, # Sazonalidade mensal
7      max_p=5, max_q=5,
8      max_P=2, max_Q=2,
9      d=1, D=1,
10     xreg=dummies_sarima, # Dummies para os outliers
11     trace=True
12 )
13 print(sarima_log.summary())

```

SARIMAX Results						
=====						
Dep. Variable:	y	No. Observations:	103			
Model:	SARIMAX(4, 1, 1)x(2, 1, [], 12)	Log Likelihood	-171.088			
Date:	Sun, 24 Nov 2024	AIC	358.17			
Time:	19:39:34	BIC	378.175			
Sample:	0	HQIC	366.241			
	- 103					
Covariance Type:	opg					
=====						
	coef	std err	z	P> z	[0.025	0.975]

ar.L1	-0.1220	0.136	-0.896	0.370	-0.389	0.145
ar.L2	-0.4842	0.137	-3.538	0.000	-0.752	-0.216
ar.L3	-0.1438	0.161	-0.892	0.372	-0.460	0.172
ar.L4	-0.3673	0.110	-3.326	0.001	-0.584	-0.151
ma.L1	-0.8503	0.119	-7.147	0.000	-1.083	-0.617
ar.S.L12	-0.6882	0.125	-5.500	0.000	-0.933	-0.443
ar.S.L24	-0.3546	0.124	-2.869	0.004	-0.597	-0.112
sigma2	2.3448	0.328	7.147	0.000	1.702	2.988
=====						
Ljung-Box (L1) (Q):	0.21	Jarque-Bera (JB):	14.05			
Prob(Q):	0.65	Prob(JB):	0.00			
Heteroskedasticity (H):	0.99	Skew:	-0.88			
Prob(H) (two-sided):	0.98	Kurtosis:	3.81			
=====						
Warnings:						
[1] Covariance matrix calculated using the outer product of gradients (complex-step).						

Figura 35: Saída do programa - Ajuste do modelo no lgo da série

- **Especificação da equação:** Após ajustar o modelo, apresentaremos os valores de p , d , q (ordens ARIMA) e P , D , Q (componentes sazonais):

O modelo obtido foi SARIMA(4,1,1)(2,1,0)[12], onde:

- $p = 4$: ordem regressiva (AR) não-sazonal;
- $d = 1$: Número de diferenciações não sazonais;
- $q = 1$: ordem de média móvel (MA) não-sazonal;
- $P = 2$: ordem autorregressiva sazonal (AR) com período sazonal de 12 meses;
- $D = 1$: número de diferenciações sazonais;
- $Q = 0$: ordem de média móvel sazonal (MA) com período sazonal de 12 meses;
- $m = 12$: período sazonal.

A equação estimada para o modelo **SARIMA(4,1,1)(2,1,0)[12]** é expressa como:

$$(1 + 0.1220L + 0.4842L^2 + 0.1438L^3 + 0.3673L^4)(1 + 0.6882L^{12})(1 + 0.3546L^{24})(1 - L)Y_t = (1 - 0.8503L)\epsilon_t$$

Este modelo possui um número menor de defasagens autorregressivas em comparação com o obtido anteriormente, sugerindo que a transformação log pode ter reduzido a complexidade do padrão não sazonal. Também requer uma diferença não sazonal para estacionariedade. Inclui um termo de média móvel, ausente no modelo sem log. A introdução de dois termos autorregressivos sazonais indica melhor captura de padrões sazonais na série log-transformada. Com menor número de termos autorregressivos não sazonais e inclusão de componentes sazonais e de média móvel,

o modelo no log parece mais compacto e teoricamente mais adaptado aos padrões sazonais.

Da mesma forma, a equação estimada para o modelo obtido anteriormente, **SARIMA(5,1,0)(0,1,0)[12]** é expressa como:

$$(1 + 1.2421L)(1 + 1.0221L^2)(1 + 0.7298L^3)(1 + 0.4522L^4)(1 + 0.1594L^5)(1 - L)Y_t = \epsilon_t$$

onde $p = 5$, $d = 1$, $q = 0$, $P = 0$, $D = 1$, $Q = 0$ e $m = 12$.

Número de defasagens autorregressivas não sazonais foi maior no modelo original e ambas as séries requerem uma diferença não sazonal para estacionariedade. Não há termo de média móvel não sazonal no modelo original. O componente sazonal é mínimo, consistente com a série log-transformada. Com cinco termos autorregressivos, o modelo apresenta mais flexibilidade para capturar padrões complexos em relação ao modelo ajustado ao log.

1.6.2 ii. Diagnósticos do Modelo

- **Diagnósticos no log da série:** O modelo ajustado ao log da série foi diagnosticado utilizando o critério de informação Akaike (*AIC*), o teste Ljung-Box para verificar a presença de autocorrelação nos resíduos, o teste Jarque-Bera para avaliar a normalidade dos resíduos, e a heteroscedasticidade foi avaliada pela estatística ARCH. Os parâmetros selecionados pelo *auto.arima* foram **SARIMAX(2, 1, 1)x(2, 1, 1, 12)**. Os resíduos apresentaram baixa autocorrelação (p -valor de Ljung-Box = 0.68), uma leve heteroscedasticidade ($H = 1.15$), e não seguiram uma distribuição normal (p -valor de Jarque-Bera = 0.00).
- **Reescalando os diagnósticos para a série original:** As previsões do modelo log-transformado foram exponenciadas (*exp*) para retornar à escala original da série. A aderência foi avaliada na escala original utilizando as métricas de erro médio absoluto (*MAE*), erro quadrático médio (*MSE*) e raiz do erro quadrático médio (*RMSE*).
- **Comparação com o modelo SARIMA ajustado à série original:** Para comparação, ajustou-se um modelo diretamente na série original, selecionando os parâmetros **SARIMAX(3, 1, 0)x(0, 1, 0, 12)**. O *AIC* do modelo log-transformado (**365.187**) foi significativamente menor que o do modelo ajustado à série original (**1449.300**), indicando uma melhor qualidade de ajuste. Além disso, as métricas de erro foram melhores para o modelo log-transformado ($MAE = 1.324$, $MSE = 7.88$, $RMSE = 2.80$) em comparação ao modelo ajustado à série original ($MAE = 8.387$, $MSE = 969.39$, $RMSE = 31.14$).

Para implementar as etapas descritas acima, utilizamos o seguinte código:

```

1 # Ajustar o modelo SARIMA com auto_arima - LOG
2 model_log = auto_arima(
3     y_train_log,
4     seasonal=True,
5     m=12, # Frequência sazonal (dados mensais)
6     max_p=3, # Limite máximo para p
7     max_q=3, # Limite máximo para q

```

```

8     max_P=2, # Limite máximo para P (sazonal)
9     max_Q=2, # Limite máximo para Q (sazonal)
10    d=1, # Diferenciação sazonal (se conhecido)
11    D=1, # Diferenciação sazonal (se conhecido)
12    trace=True, # Exibe os parâmetros testados
13    error_action='ignore', # Ignorar erros durante o ajuste
14    suppress_warnings=True, # Suprimir avisos
15    stepwise=True # Busca stepwise para eficiência
16 )
17 print(model_log.summary())
18
19
20 # Previsões e métricas para o modelo na série original
21 forecast_log_direct = model_log.predict_in_sample()
22 residuals_log_direct = y_train_log - forecast_log_direct
23
24 mae_log = mean_absolute_error(y_train_log,
25                               forecast_log_direct)
26 mse_log = mean_squared_error(y_train_log,
27                               residuals_log_direct)
28 rmse_log = np.sqrt(mse_log)
29
30 print(f"MAE (log-transformado): {mae_log}")
31 print(f"MSE (log-transformado): {mse_log}")
32 print(f"RMSE (log-transformado): {rmse_log}")
33
34 # Ajustar o modelo SARIMA com auto_arima - ORIGINAL
35 model_original = auto_arima(
36     y_train,
37     seasonal=True,
38     m=12, # Frequência sazonal (dados mensais)
39     max_p=3, # Limite máximo para p
40     max_q=3, # Limite máximo para q
41     max_P=2, # Limite máximo para P (sazonal)
42     max_Q=2, # Limite máximo para Q (sazonal)
43     d=1, # Diferenciação sazonal (se conhecido)
44     D=1, # Diferenciação sazonal (se conhecido)
45     trace=True, # Exibe os parâmetros testados
46     error_action='ignore', # Ignorar erros durante o ajuste
47     suppress_warnings=True, # Suprimir avisos
48     stepwise=True # Busca stepwise para eficiência
49 )
50 print(model_original.summary())
51
52 # Previsões e métricas para o modelo na série original
53 forecast_original_direct = model_original.predict_in_sample()
54 residuals_original_direct = y_train -
55     forecast_original_direct

```

```

55
56 mae_original = mean_absolute_error(y_train,
    forecast_original_direct)
57 mse_original = mean_squared_error(y_train,
    residuals_original_direct)
58 rmse_original = np.sqrt(mse_original)
59
60 print(f"MAE (original): {mae_original}")
61 print(f"MSE (original): {mse_original}")
62 print(f"RMSE (original): {rmse_original}")

```

```

=====
SARIMAX Results
=====
Dep. Variable:          y      No. Observations:      103
Model:                SARIMAX(2, 1, 1)x(2, 1, [], 12)  Log Likelihood    -176.594
Date:                  Sun, 24 Nov 2024              AIC              365.187
Time:                  20:36:48                      BIC              380.186
Sample:                0                            HQIC              371.236
                    - 103
Covariance Type:      opg
=====
              coef    std err          z      P>|z|      [0.025    0.975]
-----
ar.L1         -0.0123     0.130     -0.094     0.925     -0.267     0.243
ar.L2         -0.3285     0.126     -2.615     0.009     -0.575     -0.082
ma.L1         -0.9162     0.066    -13.960     0.000     -1.045     -0.788
ar.S.L12      -0.6752     0.117     -5.763     0.000     -0.905     -0.446
ar.S.L24      -0.3369     0.138     -2.444     0.015     -0.607     -0.067
sigma2         2.6676     0.355     7.522     0.000     1.973     3.363
=====
Ljung-Box (L1) (Q):           0.17  Jarque-Bera (JB):          22.90
Prob(Q):                     0.68  Prob(JB):              0.00
Heteroskedasticity (H):       1.15  Skew:                -1.02
Prob(H) (two-sided):          0.71  Kurtosis:             4.41
=====

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).

✓ # Previsões e métricas para o modelo na série original ...

MAE (original): 1.324220240182534
MSE (original): 7.886741431791492
RMSE (original): 2.8083342806353184

```

Figura 36: Saída do Programa - Ajuste do modelo com auto_arima - Com LOG

```

Best model: ARIMA(3,1,0)(0,1,0)[12]
Total fit time: 5.266 seconds

SARIMAX Results
=====
Dep. Variable: y No. Observations: 201
Model: SARIMAX(3, 1, 0)x(0, 1, 0, 12) Log Likelihood: -720.650
Date: Sun, 24 Nov 2024 AIC: 1449.300
Time: 20:44:27 BIC: 1462.246
Sample: 0 HQIC: 1454.546
Covariance Type: opg
=====
coef std err z P>|z| [0.025 0.975]
-----
ar.L1 -1.1258 0.062 -18.304 0.000 -1.246 -1.005
ar.L2 -0.7449 0.072 -10.297 0.000 -0.887 -0.603
ar.L3 -0.2946 0.069 -4.244 0.000 -0.431 -0.159
sigma2 124.1355 9.995 12.419 0.000 104.545 143.726
=====
Ljung-Box (L1) (Q): 1.17 Jarque-Bera (JB): 17.01
Prob(Q): 0.28 Prob(JB): 0.00
Heteroskedasticity (H): 2.10 Skew: -0.07
Prob(H) (two-sided): 0.00 Kurtosis: 4.47
=====

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).

✓ # Previsões e métricas para o modelo na série original ...

MAE (original): 8.387818237556118
MSE (original): 969.3951880992846
RMSE (original): 31.135111820889364

```

Figura 37: Saída do programa - Ajuste do modelo com auto_arima - Sem LOG

O modelo ajustado ao log da série apresentou melhores resultados em termos de qualidade de ajuste (AIC) e métricas de aderência (MAE , MSE , $RMSE$) em comparação ao modelo ajustado diretamente à série original. A transformação logarítmica contribuiu para estabilizar a variância e melhorar o desempenho preditivo do modelo.

Tabela 10: Comparação dos Modelos com e sem Transformação Logarítmica

Métrica	Com LOG	Sem LOG	Diferença (%)
AIC	365.187	1449.300	-74.80
MAE	1.324	8.387	-84.21
MSE	7.88	969.39	-99.19
RMSE	2.80	31.14	-91.01

1.7 Parte (g)

Enunciado: Finalmente, para os modelos SARIMA (escala original e log) e para o XGBoost, devemos efetuar a previsão 12 passos à frente, para a escala original, no período de teste. Obter as medidas de acurácia preditiva usuais.

Analisar qual o modelo que apresenta melhor desempenho preditivo no período de teste: se o modelo SARIMA para a série original, o modelo SARIMA para o log da série ou o modelo XGBoost.

Para esta parte, implementamos o seguinte código:

```

1 ##### PARTE G
2 # Fun o para calcular MAPE

```

```

3 def mean_absolute_percentage_error(y_true, y_pred):
4     return np.mean(np.abs((y_true - y_pred) / y_true)) * 100
5
6
7 # Garantir alinhamento de X_test com os últimos 12 passos
8 X_test_adjusted = X_test.tail(12)
9
10 # Garantir alinhamento de y_test com as previsões
11 y_test_adjusted = y_test[-12:]
12
13 # Previsões dos modelos
14 sarima_original_forecast = model_original.predict(n_periods
15     =12)
16 sarima_log_forecast = np.exp(
17     model_log.predict(n_periods=12)) # Reescalando o log
18 xgboost_forecast = xgb_model.predict(X_test_adjusted)
19
20 # Calcular métricas para SARIMA (escala original)
21 rmse_sarima_original = np.sqrt(mean_squared_error(
22     y_test_adjusted, sarima_original_forecast))
23 mae_sarima_original = mean_absolute_error(
24     y_test_adjusted, sarima_original_forecast)
25 mape_sarima_original = mean_absolute_percentage_error(
26     y_test_adjusted, sarima_original_forecast)
27
28 # Calcular métricas para SARIMA (log)
29 rmse_sarima_log = np.sqrt(mean_squared_error(
30     y_test_adjusted, sarima_log_forecast))
31 mae_sarima_log = mean_absolute_error(y_test_adjusted,
32     sarima_log_forecast)
33 mape_sarima_log = mean_absolute_percentage_error(
34     y_test_adjusted, sarima_log_forecast)
35
36 # Calcular métricas para XGBoost
37 rmse_xgboost = np.sqrt(mean_squared_error(y_test_adjusted,
38     xgboost_forecast))
39 mae_xgboost = mean_absolute_error(y_test_adjusted,
40     xgboost_forecast)
41 mape_xgboost = mean_absolute_percentage_error(
42     y_test_adjusted, xgboost_forecast)
43
44 # Exibir resultados consolidados
45 print("Métricas de Acurácia no Período de Teste:")
46 print("SARIMA Original - RMSE:", rmse_sarima_original, "MAE:",
47     ,
48     mae_sarima_original, "MAPE:", mape_sarima_original)
49 print("SARIMA Log - RMSE:", rmse_sarima_log, "MAE:",
50     mae_sarima_log, "MAPE:", mape_sarima_log)
51 print("XGBoost - RMSE:", rmse_xgboost, "MAE:",
52     mae_xgboost, "MAPE:", mape_xgboost)

```

```
Métricas de Acurácia no Período de Teste:
SARIMA Original - RMSE: 51.326293111130774 MAE: 42.92565394685136 MAPE: 947.3905457020587
SARIMA Log - RMSE: 32.29695655446462 MAE: 26.61579564438715 MAPE: 346.8092238756361
XGBoost - RMSE: 27.766076264941404 MAE: 18.603485368531548 MAPE: 96.85225414024711
```

Figura 38: Saída do programa - Métricas de Acurácia - 1.G

1.7.1 Resultados Obtidos

As métricas de acurácia preditiva para os três modelos ajustados foram calculadas e apresentadas abaixo:

- **SARIMA (escala original):**
 - *RMSE (Raiz do Erro Quadrático Médio):* 51.33
 - *MAE (Erro Médio Absoluto):* 42.93
 - *MAPE (Erro Percentual Absoluto Médio):* 947.39%
- **SARIMA (log-transformado):**
 - *RMSE:* 32.30
 - *MAE:* 26.62
 - *MAPE:* 346.81%
- **XGBoost:**
 - *RMSE:* 27.77
 - *MAE:* 18.60
 - *MAPE:* 96.85%

1.7.2 Análise e Interpretação

Os resultados revelam importantes diferenças de desempenho entre os modelos, que serão analisadas a seguir:

- **SARIMA (escala original):** Este modelo apresentou os maiores valores de RMSE (51.33), MAE (42.93) e MAPE (947.39%). O elevado valor de MAPE indica que, embora o modelo capture parte das variações absolutas, ele falha em representar proporcionalmente os valores reais da série. Isso pode ser atribuído à grande variação de escala nos dados originais, sem tratamento para estabilizar a variância.
- **SARIMA (log-transformado):** A transformação logarítmica trouxe melhorias significativas em comparação ao modelo na escala original. O RMSE foi reduzido em aproximadamente 37% e o MAE em cerca de 38%. No entanto, o MAPE (346.81%) ainda é elevado, indicando que o modelo não consegue capturar pequenas variações relativas da série no período de teste. A transformação log parece ter estabilizado a variância da série, melhorando o desempenho preditivo, mas ainda não foi suficiente para reduzir o erro percentual médio a níveis aceitáveis.

- **XGBoost**: Este modelo apresentou o melhor desempenho geral, com os menores valores de RMSE (27.77), MAE (18.60) e MAPE (96.85%). Em particular, o valor de MAPE, significativamente menor do que os dos modelos SARIMA, evidencia que o XGBoost foi mais eficaz em capturar as variações relativas da série. Isso pode ser atribuído à capacidade do XGBoost de lidar com dados complexos e modelar não linearidades, enquanto os modelos SARIMA dependem de uma estrutura paramétrica fixa.

1.7.3 Conclusão da parte 1.G

Com base nos resultados obtidos, o modelo **XGBoost** demonstrou o melhor desempenho preditivo no período de teste, com menor erro absoluto e percentual em comparação aos modelos SARIMA. O uso de técnicas baseadas em aprendizado de máquina mostrou-se vantajoso para capturar padrões mais complexos presentes na série temporal, mesmo quando comparado ao SARIMA com transformação logarítmica. No entanto, a escolha do modelo deve considerar o contexto da aplicação e a interpretabilidade, uma vez que os modelos SARIMA oferecem maior transparência nos processos de previsão.

2 Segunda Questão

Enunciado: Considere agora toda a amostra. Re-estime o melhor modelo SARIMA identificado em g) e o modelo XGBoost, obtendo tabela com valores previstos 24 meses à frente.

O melhor modelo SARIMA identificado na questão G foi o modelo SARIMA no log da série. Ele apresentou as seguintes métricas de acurácia no período de teste:

- RMSE: 32.39
- MAE: 26.62
- MAPE: 346.81

Comparado ao modelo SARIMA na escala original (que teve RMSE de 51.33, MAE de 42.93 e MAPE de 947.39), o modelo ajustado ao log foi significativamente superior em termos de desempenho preditivo, indicando maior aderência aos dados. Portanto, na questão 2, utilizaremos o modelo SARIMA ajustado ao log da série para reestimar com toda a amostra e prever os 24 passos à frente.

2.0.1 Código Python Utilizado

O código abaixo reestima o melhor modelo SARIMA (identificado na questão g como o modelo aplicado ao log da série) e o modelo XGBoost, utilizando toda a amostra para previsão de 24 meses à frente.

```
1 ##### QUEST 0 2 #####
2
3 # Aplicando log na s rie completa
4 y_full_log = np.log(y)
5 y_full_log = y_full_log[~np.isnan(y_full_log)] # Removendo
   valores NaN
6
7 # Ajustar o melhor modelo SARIMA com toda a amostra
8 model_sarima_full = auto_arima(
9     y_full_log,
10    seasonal=True,
11    m=12, # Sazonalidade mensal
12    max_p=3, max_q=3,
13    max_P=2, max_Q=2,
14    d=1, D=1,
15    trace=True,
16    stepwise=True
17 )
18 print(model_sarima_full.summary())
19
20 # Previs es para 24 passos      frente (SARIMA)
21 sarima_forecast_24 = model_sarima_full.predict(n_periods=24)
22
23 # Criar lags e dummies para o conjunto completo
24 n_lags = 3 # N mero de defasagens utilizadas
```

```

25 y_lagged_full = create_lags(y, n_lags)
26
27 # Dummies para os meses de outliers
28 dummies_full = pd.DataFrame(
29     np.zeros((len(y_lagged_full), len(outlier_months_xgboost)
30         )),
31     columns=[f'outlier_{month}' for month in
32         outlier_months_xgboost]
33 )
34 for i, month in enumerate(outlier_months_xgboost):
35     if month < len(dummies_full):
36         dummies_full.iloc[month, i] = 1
37
38 X_full = pd.concat([y_lagged_full, dummies_full], axis=1)
39 X_full = X_full.astype(float) # Garantir que todos os dados
    sejam float
40
41 # Ajustar o XGBoost com a amostra completa
42 xgb_model_full = XGBRegressor(n_estimators=100, max_depth=3,
43     learning_rate=0.1)
44 xgb_model_full.fit(X_full, y[n_lags:])
45
46 # Previsões para 24 passos frente (XGBoost)
47 X_future = create_future_features(
48     y, n_lags, 24, outlier_months_xgboost, X_full.columns
49 )
50 X_future = X_future.astype(float) # Garantir que todos os
    dados sejam float
51 xgboost_forecast_24 = xgb_model_full.predict(X_future)
52
53 # Criar tabela de resultados
54 if isinstance(y, np.ndarray):
55     y = pd.Series(y, index=pd.date_range(start="2000-01-01",
56         periods=len(y), freq="M"))
57
58 # Gerar datas para as previsões
59 months = pd.date_range(start=y.index[-1] + pd.offsets.
60     MonthEnd(1), periods=24, freq='M')
61
62 # Criar a tabela de previsões
63 forecast_table = pd.DataFrame({
64     "M s": months,
65     "SARIMA": sarima_forecast_24,
66     "XGBoost": xgboost_forecast_24
67 })
68
69 # Exibir a tabela de previsões
70 print(forecast_table)

```

2.0.2 Resultados Obtidos

```

✓ print(model_sarima_full.summary()) ...

=====
SARIMAX Results
=====
Dep. Variable:          y      No. Observations:      123
Model:      SARIMAX(0, 1, 3)x(0, 1, [1], 12)      Log Likelihood      -189.424
Date:              Mon, 25 Nov 2024      AIC      390.847
Time:              00:56:43      BIC      407.050
Sample:              0      HQIC      397.419
              - 123
Covariance Type:      opg
=====
              coef      std err      z      P>|z|      [0.025      0.975]
-----
intercept      -0.0097      0.004      -2.597      0.009      -0.017      -0.002
ma.L1           -1.0096      0.104      -9.735      0.000      -1.213      -0.806
ma.L2           -0.3075      0.142      -2.159      0.031      -0.587      -0.028
ma.L3            0.3802      0.098      3.877      0.000      0.188      0.572
ma.S.L12        -0.7309      0.105      -6.961      0.000      -0.937      -0.525
sigma2           1.6388      0.253      6.478      0.000      1.143      2.135
=====
Ljung-Box (L1) (Q):      0.00      Jarque-Bera (JB):      0.70
Prob(Q):      0.96      Prob(JB):      0.70
Heteroskedasticity (H):      1.12      Skew:      -0.01
Prob(H) (two-sided):      0.73      Kurtosis:      3.39
=====

```

Figura 39: Saída do Programa - Modelo SARIMA ajustado (com log)

	Mês	SARIMA	XGBoost
123	2021-05-31	1.591963	-11.277020
124	2021-06-30	2.102739	2.730706
125	2021-07-31	2.627202	1.777173
126	2021-08-31	1.208302	-12.373295
127	2021-09-30	0.608424	2.276542
128	2021-10-31	0.752581	1.777173
129	2021-11-30	1.287747	-12.468056
130	2021-12-31	1.956602	2.233166
131	2022-01-31	2.495557	1.733796
132	2022-02-28	1.816977	-12.468056
133	2022-03-31	0.690897	2.207579
134	2022-04-30	0.927792	1.477872
135	2022-05-31	0.966091	-12.511432
136	2022-06-30	1.340465	2.207579
137	2022-07-31	2.028244	1.320212
138	2022-08-31	0.599615	-12.468056
139	2022-09-30	-0.009992	2.207579
140	2022-10-31	0.124436	1.320212
141	2022-11-30	0.649873	-12.468056
142	2022-12-31	1.308999	2.164202
143	2023-01-31	1.838224	1.276835
144	2023-02-28	1.149916	-12.468056
145	2023-03-31	0.014106	2.207579
146	2023-04-30	0.241272	1.193075

Figura 40: Saída do Programa - Forecast Table

2.0.3 Análise dos Resultados

Modelo SARIMA: O modelo SARIMA, ajustado ao log da série, gerou previsões razoavelmente consistentes e positivas para os próximos 24 meses. A transformação logarítmica foi revertida, resultando em valores previsíveis e sem discrepâncias graves.

Modelo XGBoost: O modelo XGBoost, por outro lado, apresentou valores negativos em algumas previsões, especialmente nos períodos iniciais. Essa inconsistência pode estar relacionada a problemas de escalonamento ou à modelagem inadequada de variáveis categóricas ou lags. Ainda assim, em alguns meses, o modelo produziu valores positivos coerentes.

Comparação: O modelo SARIMA apresentou melhor desempenho em termos de consistência e adequação às tendências da série temporal. O modelo XGBoost, embora útil em muitas aplicações, mostrou limitações ao lidar com as características específicas da série e variáveis.

2.0.4 Conclusões Gerais da Questão 2

- O modelo SARIMA foi o mais confiável para a previsão de 24 meses à frente, especialmente após a transformação logarítmica.
- O modelo XGBoost requer ajustes adicionais, como refinamento no tratamento de variáveis categóricas e escalonamento.

Os resultados indicam que o modelo SARIMA, quando ajustado corretamente, é uma escolha sólida para séries temporais sazonais e log-transformadas.